

- [[#Лекция 1 (7.02.26)|Лекция 1 (7.02.26)]]
 - [[#Лекция 1 (7.02.26)#Параграф 1.1 - Модели архитектур вычислительных систем|Параграф 1.1 - Модели архитектур вычислительных систем]]
- [[#Лекция 2 (14.02.26)|Лекция 2 (14.02.26)]]
 - [[#Лекция 2 (14.02.26)#Параграф 1.2 - Модели алгоритмов|Параграф 1.2 - Модели алгоритмов]]
 - [[#Лекция 2 (14.02.26)#Параграф 1.3 - Модели вычислительных процессов|Параграф 1.3 - Модели вычислительных процессов]]
- [[#Лекция 3 (21.02.26)|Лекция 3 (21.02.26)]]
- [[#Лекция 4 (28.02.26)|Лекция 4 (28.02.26)]]
 - [[#Лекция 4 (28.02.26)#Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ|Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ]]
 - * [[#Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ#Этап I - Построение стандартного графа|Этап I - Построение стандартного графа]]
 - * [[#Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ#Этап II - Построение графа зависимостей|Этап II - Построение графа зависимостей]]
 - * [[#Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ#Этап III - Построение ярусно параллельной формы|Этап III - Построение ярусно параллельной формы]]
 - * [[#Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ#Этап IV - Построение параллельного алгоритма|Этап IV - Построение параллельного алгоритма]]
 - * [[#Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ#Формализм определения функции $c(A)$ |Формализм определения функции $c(A)$]]
- [[#Лекция 5 (07.03.26)|Лекция 5 (07.03.26)]]
 - [[#Лекция 5 (07.03.26)#Параграф 2.2 Автоматическое распараллеливание циклических фрагментов последовательной программы|Параграф 2.2 Автоматическое распараллеливание циклических фрагментов последовательной программы]]
 - * [[#Параграф 2.2 Автоматическое распараллеливание циклических фрагментов последовательной программы#Пункт 2.2.1 - Метод пирамид|Пункт 2.2.1 - Метод пирамид]]
- [[#Лекция 6 (14.03.26)|Лекция 6 (14.03.26)]]
 - [[#Лекция 6 (14.03.26)#Пункт 2.2.2 Метод параллелепипедов|Пункт 2.2.2 Метод параллелепипедов]]
- [[#Лекция 7 (21.03.26)|Лекция 7 (21.03.26)]]
 - [[#Лекция 7 (21.03.26)#Параграф 3.1 Векторные алгоритмы умножения матриц|Параграф 3.1 Векторные алгоритмы умножения матриц]]
 - * [[#Параграф 3.1 Векторные алгоритмы умножения матриц#Пункт 3.1.1 Основные векторные операции|Пункт 3.1.1 Основные векторные операции]]
 - * [[#Параграф 3.1 Векторные алгоритмы умножения матриц#Пункт 3.1.2 Векторные алгоритмы умножения матриц|Пункт 3.1.2 Векторные алгоритмы умножения матриц]]
 - [[#Лекция 7 (21.03.26)#Параграф 3.2 Векторные алгоритмы решения СЛАУ|Параграф 3.2 Векторные алгоритмы решения СЛАУ]]
 - * [[#Параграф 3.2 Векторные алгоритмы решения СЛАУ#Пункт 3.2.1 Решение СЛАУ с матрицами треугольного вида|Пункт 3.2.1 Решение СЛАУ с матрицами треугольного вида]]
- [[#Лекция 8 (28.03.26)|Лекция 8 (28.03.26)]] - [[#Параграф 3.2 Векторные алгоритмы решения СЛАУ#Пункт 3.2.2 Алгоритм LU разложения через модификацию внешним произведением|Пункт 3.2.2 Алгоритм LU разложения через модификацию внешним произведением]] - [[#Параграф 3.2 Векторные алгоритмы решения СЛАУ#Пункт 3.2.3 Производственное LU разложение через гаусс|Пункт 3.2.3 Производственное LU разложение через гаусс]]
- [[#Лекция 9 (04.04.2026)|Лекция 9 (04.04.2026)]] - [[#Параграф 3.2 Векторные алгоритмы решения СЛАУ#Пункт 3.2.4 Блочный алгоритм LU разложения|Пункт 3.2.4 Блочный алгоритм LU разложения]]
 - [[#Лекция 9 (04.04.2026)#Параграф 3.3 Разложение Холецкого|Параграф 3.3 Разложение Холецкого]]
 - * [[#Параграф 3.3 Разложение Холецкого#Пункт 3.3.1 Разложение Холецкого через модификацию матрицы внешним произведением|Пункт 3.3.1 Разложение Холецкого через модификацию матрицы внешним произведением]]
- [[#Лекция 10 (11.04.26)|Лекция 10 (11.04.26)]]
 - [[#Лекция 10 (11.04.26)#Пункт 3.3.2 Разложение Холецкого через гаусс|Пункт 3.3.2 Разложение Холецкого через гаусс]]
 - [[#Лекция 10 (11.04.26)#Пункт 3.3.3 Блочные алгоритмы метода Холецкого|Пункт 3.3.3 Блочные алгоритмы метода Холецкого]]
 - * [[#Пункт 3.3.3 Блочные алгоритмы метода Холецкого#Пункт I Блочный алгоритм Холецкого на основе

- ???[Пункт I Блочный алгоритм Холецкого на основе ???]
- * [[#Пункт 3.3.3 Блочные алгоритмы метода Холецкого#Пункт IIa|Пункт IIa]]
- * [[#Пункт 3.3.3 Блочные алгоритмы метода Холецкого#Пункт IIб|Пункт IIб]]
- * [[#Пункт 3.3.3 Блочные алгоритмы метода Холецкого#Пункт IIс|Пункт IIс]]
- [[#Лекция 11 (18.04.2026)|Лекция 11 (18.04.2026)]]
 - [[#Лекция 11 (18.04.2026)#Параграф 4.1 Теоретическое и экспериментальное исследование параллельных алгоритмов|Параграф 4.1 Теоретическое и экспериментальное исследование параллельных алгоритмов]]
 - * [[#Параграф 4.1 Теоретическое и экспериментальное исследование параллельных алгоритмов#Пункт 4.1.1 Эффективность, ускорение, масштабируемость|Пункт 4.1.1 Эффективность, ускорение, масштабируемость]]
 - * [[#Параграф 4.1 Теоретическое и экспериментальное исследование параллельных алгоритмов#Пункт 4.1.2|Пункт 4.1.2]]
 - [[#Пункт 4.1.2#Пункт 4.1.2.1 Закон Амдала (1957)|Пункт 4.1.2.1 Закон Амдала (1957)]]
 - [[#Пункт 4.1.2#Пункт 4.1.2.2 Учет коммуникаций|Пункт 4.1.2.2 Учет коммуникаций]]
 - * [[#Параграф 4.1 Теоретическое и экспериментальное исследование параллельных алгоритмов#Пункт 4.1.3 Экспериментальные исследования параллельных алгоритмов|Пункт 4.1.3 Экспериментальные исследования параллельных алгоритмов]]

Глава 1 - Модели в теории параллельных вычислений

Лекция 1 (7.02.26)

Г.И. Марчук сформулировал: Это отображение численного метода на архитектуру вычислительной системы. Верно и обратное.

Инвариант Янинко - каким бы не был уровень развития математики, каким бы не был уровень развития эвм???

Уровни сложности:

1. Автоматическое распараллеливание
2. Последовательный алгоритм → параллельный алгоритм
3. Параллельный численный метод

А.А. Самарский - Триада Самарского Объект → мат.модель → численные методы → программа (Мат. модель никогда не описывает объект полностью)

Параграф 1.1 - Модели архитектур вычислительных систем

Модель (классификация) Флинна 1966 г.

данные, команды → ЭВМ

команды/потоки	одна команда	много команд
один поток	однопроц. ЭВМ (SISD)	конвейерные ЭВМ (MISD)
много потоков	векторные (SIMD)	многопроц. ЭВМ (MIMD)

Классификация Хокни для MIMD

1. Конвейерные (!Проблема взаимного исключения!)
2. Переключаемые
 1. С одной памятью
 2. С распределенной памятью
3. Сети
 1. Процессорная линия (частный случай конвейерного)
 2. Процессорное кольцо
 3. Процессорная решетка
 4. Тор (бублик)
 5. Гиперкуб
 6. Бинарное дерево (частный случай гиперкуба)
 7. Звезда
 8. Гиперкуб с размерностью k содержит $p = 2^k$ вершин

Лекция 2 (14.02.26)

1. Иерархическая структура памяти ЭВМ Пирамида (Скорость растет снизу вверх, Объем растет сверху вниз):

1. Регистры
2. Кэш процессора ((L1,L1),(L2,L2),L3)
3. Оперативная память
4. Дисковая память

tilig (блочность)

2. Модели коммуникации - коммуникация между компьютерами по сети

1. Самая простая - **Линейная модель** Время и скорость передачи непрерыв
2. Пакетная модель коммуникации

Есть время установки Время отправки одного пакета

Рекомендации по организации пакет: При выборе частой отправки небольшого количества данных или редкой отправки большого, предпочтительнее второй вариант, как более соответствующий пакетной модели

Можно аппроксимировать до линейной модели если $v \gg v_n$

Параграф 1.2 - Модели алгоритмов

По Маркову под алгоритмом будем понимать ясные предписания, задающие вычислительный процесс, идущий от варьируемых начальных данных к искомому результату

Свойства алгоритмов: - **Детерминированность (результативность)** - за конечное количество шагов алгоритм либо приводит к результату, либо к убежденности невозможности получить результат. - **Дискретность** - подразумевает, что алгоритм должен разбиваться на шаги - **Масса** - схожие задачи должны допускать решение одним алгоритмом - **Однозначность** - алгоритм не допускает двух разных трактовок

Модели параллельных алгоритмов:

1. Задача/Канал

Под **задачей** будем понимать последовательный набор инструкций и данных, над которыми инструкции выполняются (Кружочки)

Параллельность обеспечивается за счет того, что одновременно выполняются несколько задач

Канал - средство коммуникации между задачами (Стрелочки)

Два правила каналов:

1. Перемещение данных по каналу всегда в одну сторону
2. Данные по каналу перемещаются в соответствии с правилом - первый вошел, первый вышел (Вагончики)

В ходе работы алгоритма задачи могут появляться и упраздняться

Свойства коммуникации:

1. Синхронная/Асинхронная коммуникация

Операции приема и отправки

Под **синхронной коммуникацией** будем понимать ситуацию, когда следующая за этой операцией инструкция не выполняется до окончания коммуникации

В **асинхронном** случае следующая инструкция после коммуникации выполняется сразу после начала этой коммуникации

2. Локальные/Глобальные коммуникации

p - число задач

соседи $\leq \lceil \log_2 p \rceil$ до ближайшего большего числа, то локальная

3. Статические/Динамические коммуникации

Если топология коммуникации алгоритма меняется по ходу его выполнения, то такие коммуникации называются динамические

4. Регулярные/Нерегулярные коммуникации

Если коммуникация алгоритма не отображается на стандартную сетевую топологию, то она нерегулярная

2. Джин Голуб

Модель основана на аннотации языка Маркова + Операторы коммуникации Операторы: - send (A, let), где A - что посылаем, let - куда посылаем

Линия и кольцо

p - число задач

$1 \leq \mu \leq p$ - число процессов

let - $\mu - 1$

right - $\mu + 1$

Решетка и тор

(μ, λ) - номер задачи

north $(\mu - 1, \lambda)$, south $(\mu + 1, \lambda)$, west $(\mu, \lambda - 1)$, east

$(\mu, \lambda + 1)$

- recv(A, let), где A - куда принимаем, let - от кого принимаем

При записи параллельного алгоритма каждой задаче должны быть поставлены в соответствии инструкции, которая она выполняет, и данные, над которыми она работает

По Голобу алгоритму предваряется инициализация

Инициализация - p число задач, μ или $\mu - 1$ - номер задачи, локальные данные

У Голоба нет записи глобальных алгоритмов

Линейное распределение

Есть массив $a(1 : n)$, $r = \frac{n}{p}$

В ходу инициализации $\mu - a((\mu - 1), r + 1)$

```
if mu = 1 then
  for i = 2 : p
    send(a((i-1)r+1:n), i)
  endfor
else
  recv(a(1:n), 1)
endif
```

Циклическое распределение

$\mu - a(\mu : p : n)$

Фоточка

Параграф 1.3 - Модели вычислительных процессов

Вычислительный процесс - это действия по алгоритму на определенной вычислительной системе некоторое время

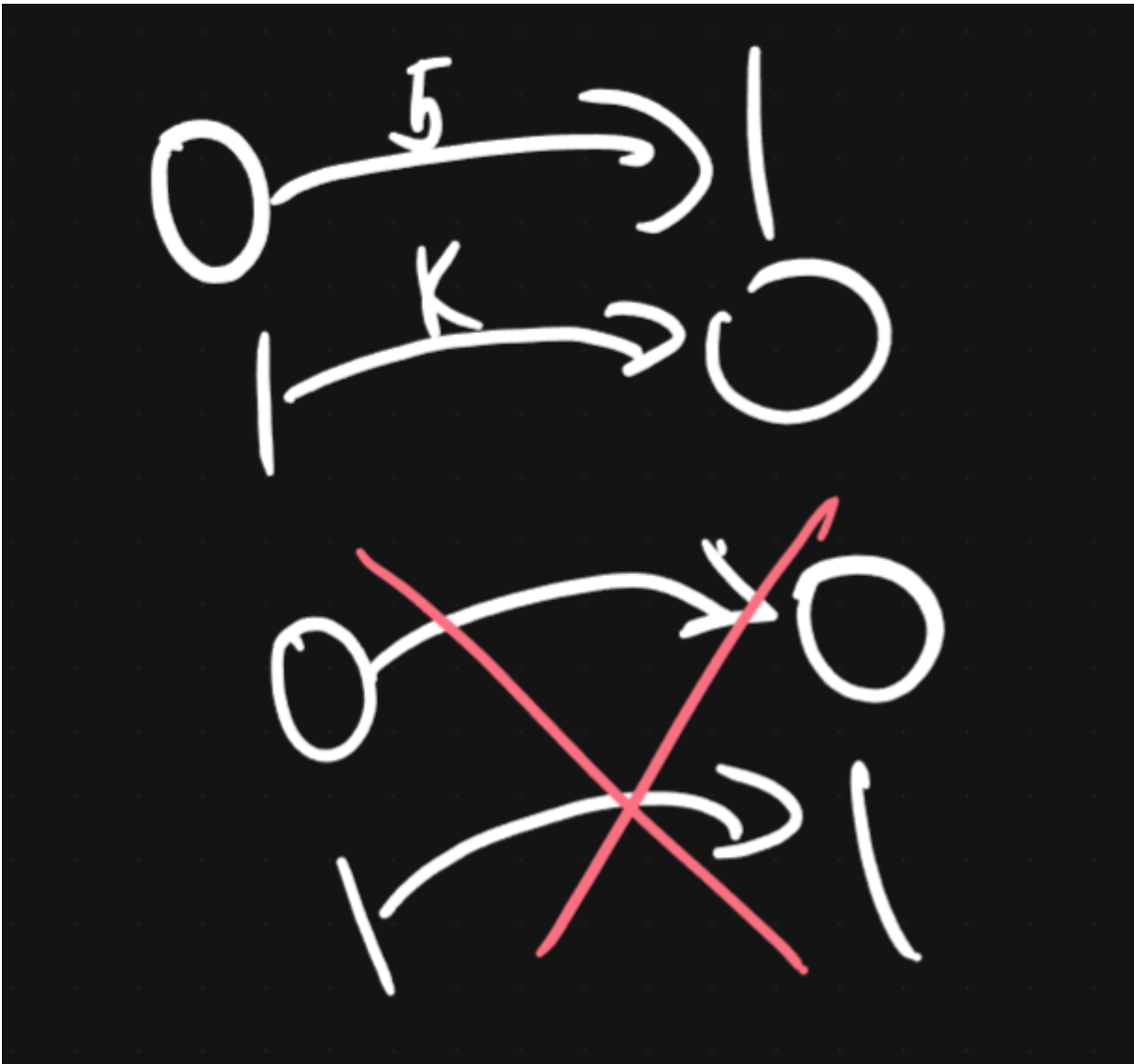
Лекция 3 (21.02.26)

Сетью назовем - $N = (P, T, F)$

P - непустое конечное множество мест (Круги на графе)

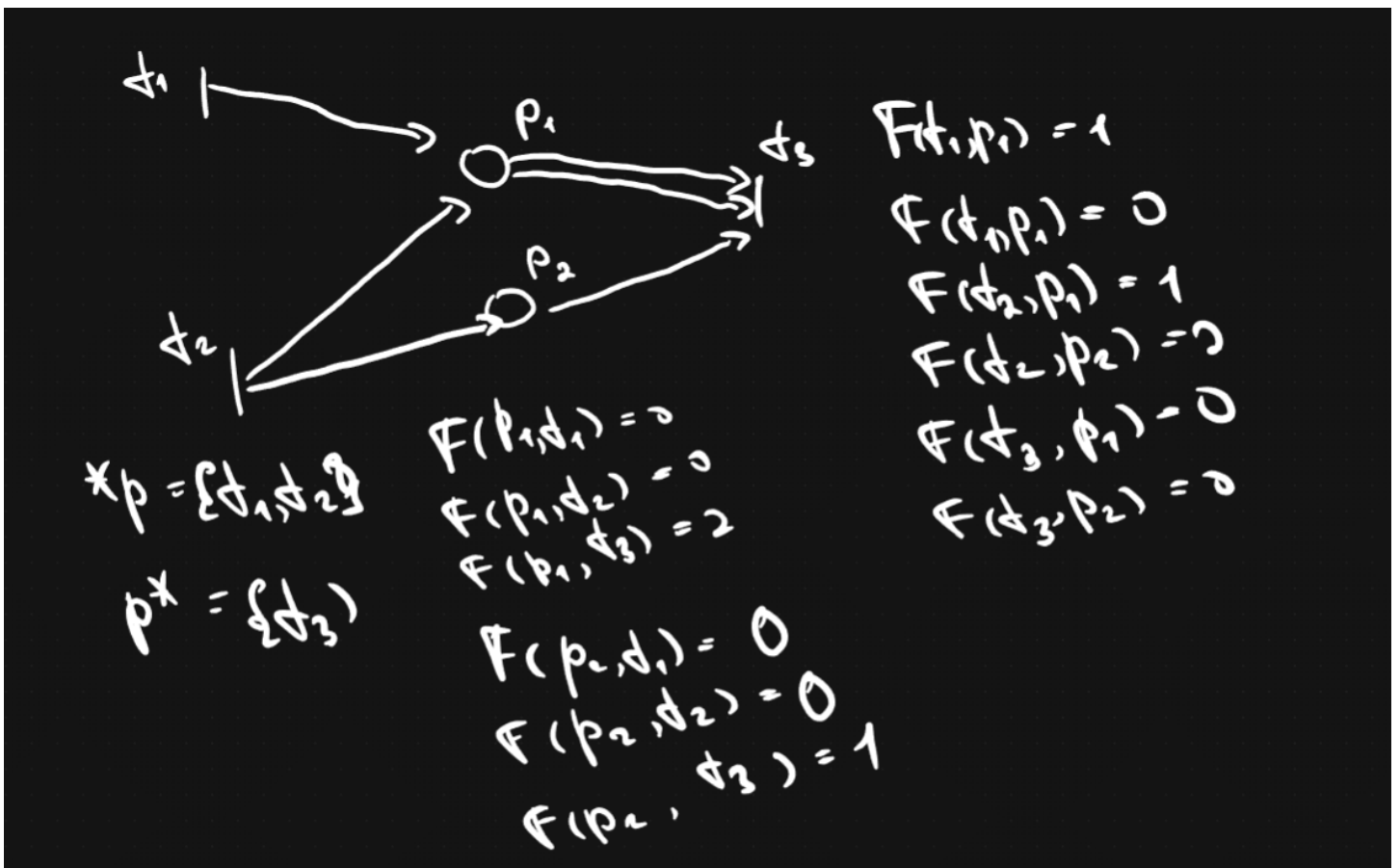
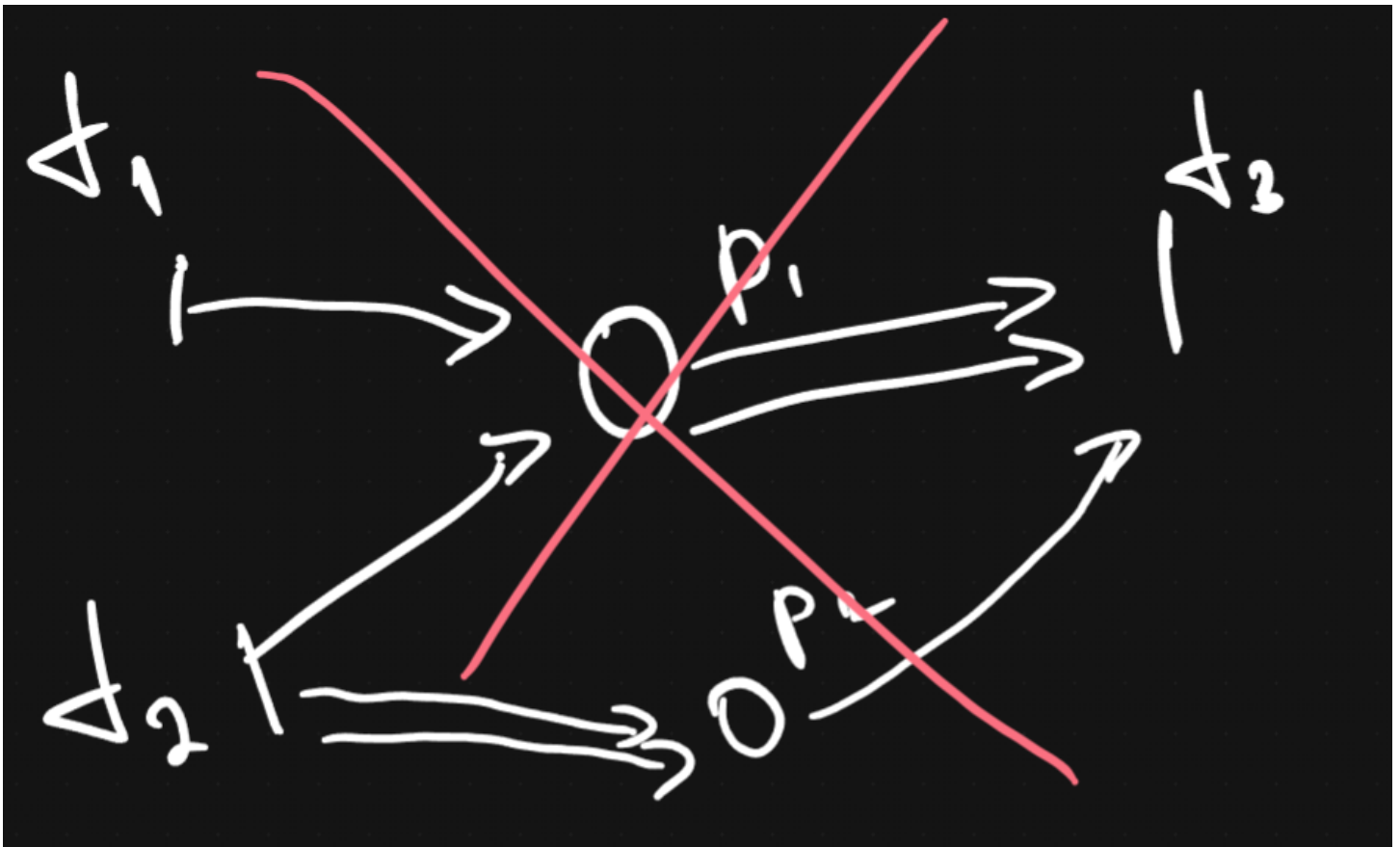
T - непустое конечное множество переходов (Отрезки на графе)

F - функция инцидентности, имеет область определения: $F : P \times T \cup T \times P \rightarrow N$



Свойства сети:

1. Переходы и места разные множества $P \cap T = \emptyset$
 2. $\forall p \in P \exists t \in T$ либо $F(p, t) \neq 0$ либо $F(t, p) \neq 0$
 3. $\forall t \in T \exists p \in P$ либо $F(t, p) \neq 0$ либо $F(p, t) \neq 0$
 4. $\forall p_1, p_2 \in P$ $(*p_1 = *p_2) \vee (p*_1 = p*_2) \rightarrow p_1 = p_2$
- где $*p$ — множество входных переходов p , p^* — множество выходных переходов p



Сеть Петри $PN = (P, T, F, M)$

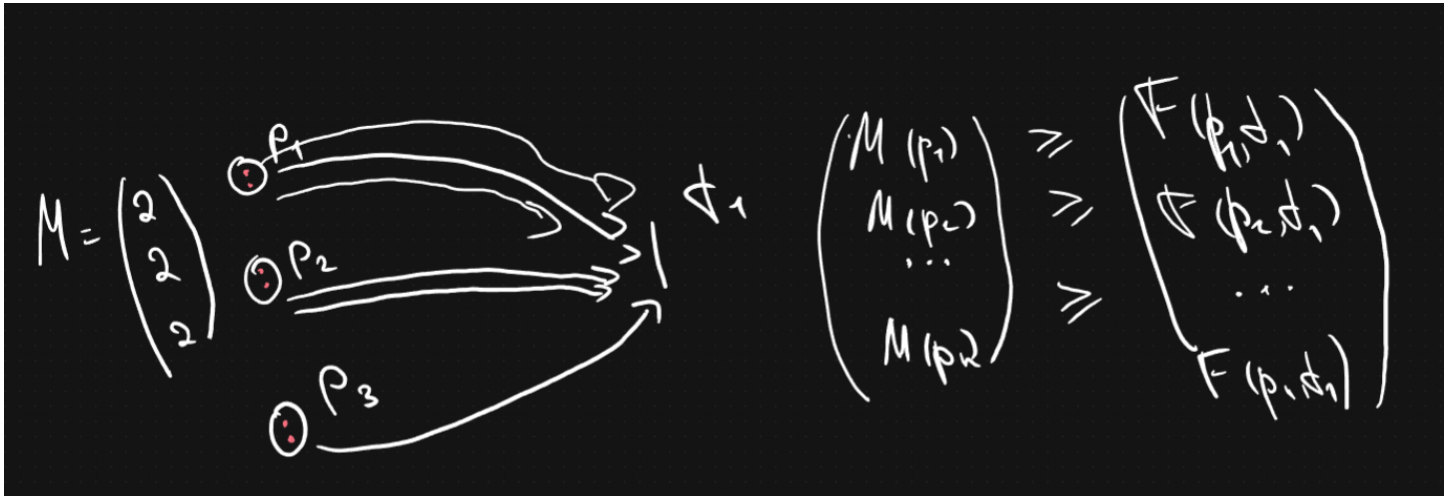
M - множество ($M : P \rightarrow \mathbb{N}$)

$$M = \begin{pmatrix} M(p_1) \\ M(p_2) \\ \dots \\ M(p_n) \end{pmatrix}$$

разметка меняется после срабатывания переходов.

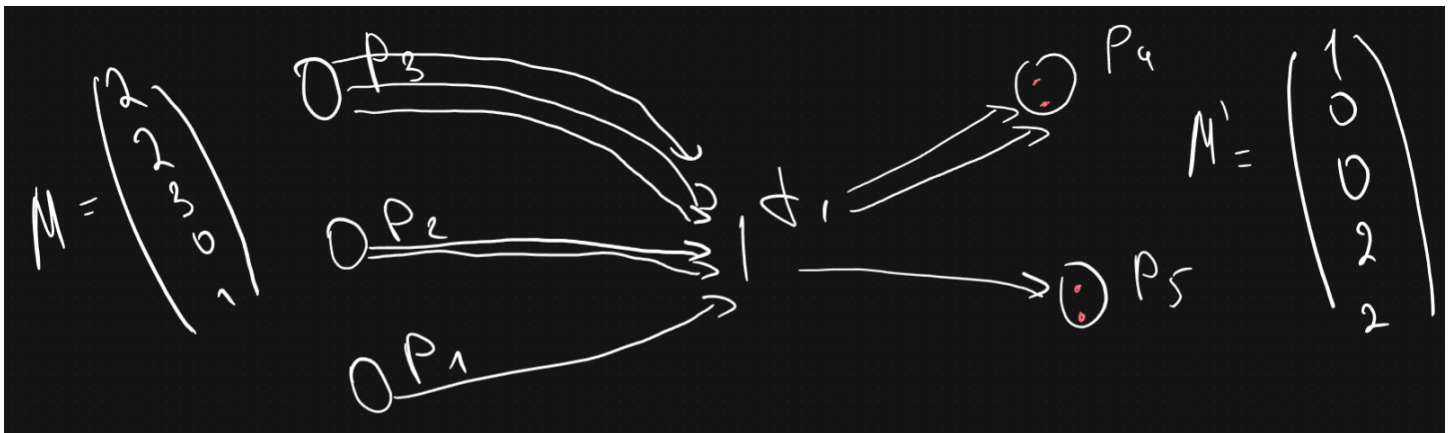
Условия срабатывания переходов: 1. Переход t - может (но не должен) сработать, если $\forall p \in {}^*t M(p) \geq F(p, t)$ в векторном виде $M \geq {}^*F(t)$

где t^* - множество входных мест перехода t

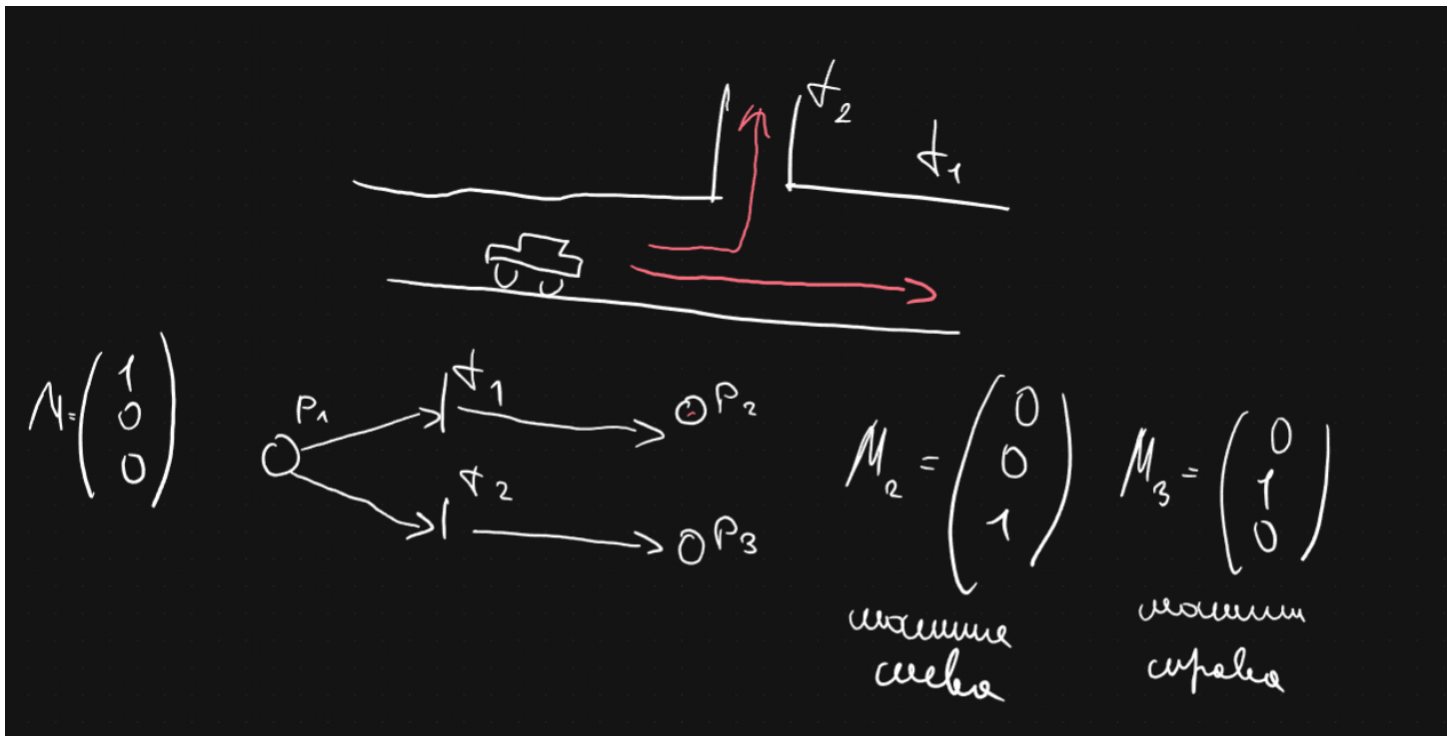


1. Допустим мог сработать и сработал, то меняется разметка $\forall p \in PM'(p) = M(p) - F(p, t) + F(t, p)$, в векторном виде $M' = M - {}^*F(t) + F^*(T)$

$$\begin{pmatrix} M(p_1) \\ M(p_2) \\ \dots \\ M(p_n) \end{pmatrix} = \begin{pmatrix} M(p_1) \\ M(p_2) \\ \dots \\ M(p_n) \end{pmatrix} - \begin{pmatrix} F(p_1, t) \\ F(p_2, t) \\ \dots \\ F(p_n, t) \end{pmatrix} + \begin{pmatrix} F(t, p_1) \\ F(t, p_2) \\ \dots \\ F(t, p_n) \end{pmatrix}$$



Пример:



Закона сохранения фишек **НЕТ!**

Работа всей сети Петри описывается:

1. Множество сработавших переходов $\tau = \{t_1, t_2 \dots\}$
2. Множество достижимых разметок $R_0 = (PN, M_0)$

$$M_0[t_1 > M_1[t_2 > M_2 \dots M_N$$

$M_0[t_1 > M_1$ - бинарное отношение непосредственно на множестве разметок

M_0, M_1 вступают в бинарное отношение непосредственного следования, если:

$$(M_0 \rightarrow *F(t_1)) \text{ и } (M_1 = M_0 - *F(t_1) + F^*(t_1))$$

Глава 2 - Автоматическое распараллеливание последовательных программ

Лекция 4 (28.02.26)

Параграф 2.1 - Автоматическое распараллеливание ациклических участков последовательных программ

Любой указанный фрагмент допускает запись через операторы всего двух видов: - Присваивание - Условные переходы

Этап I - Построение стандартного графа Будем различать два типа вершин стандартного графа:

1. **Преобразователь** - каждый соответствует одному или нескольким последовательно расположенных операторов присваивания, изображается на графе **прямоугольником**
2. **Распознаватель** - каждый соответствует строго одному оператору условного перехода, изображается на графе **окружностью**

Дуги на графе задаются бинарным отношением непосредственного следования

Отметим, что из преобразователя выходит не более одной дуги

Из распознавателя не более двух, при этом дугу соответствующей истинности условия распознавателя маркируем 1, ложное 0

Вершину в которую не входит ни одна дуга именуем **входной** вершиной стандартного графа

Вершину из которой не выходит не одна дуга именуем **выходной** вершиной стандартного графа

Пример:

A: $\text{beg}(x, y)$
 B: $l = x$
 C: $k = y$
 D: $v = c + y$
 E: $z = c + x$
 P: Если $x > y$, то на F, иначе G
 F: $l = x; l = y$
 G: $\text{переменная}(\min(z, v), \max(z, v))$
 H: $\text{связать}(l, k)$



Этап II - Построение графа зависимостей

1. Вершина В информационно зависит от А, если {А и В находятся на одном пути стандартного графа из входной вершины в выходную (при чем А предшествует В)} и хотя бы один оператор из В использует значение переменной формируем хотя бы одним оператором из А. При этом отсутствует расположенная на одном пути, такая что меняется значение

При построение графа зависимости в него переносятся вершины стандартного графа без изменений

2. Вершина В логически зависит от А, если {...} и существует другой путь на стандартном графе из входной вершины в выходную. Совпадающий с первым до вершины А включительно, далее отличающуюся от него и не содержащую В.
3. Вершина В конкурентно зависит от вершины А, если {...} и в каждой из вершин есть оператор присваивания меняющий значение некой общей переменной.

Отныне не различаем тип зависимости. Завершаем построение графа зависимости нанесение на него транзитивного замыкания зависимостей построенных к этому моменту.



→ инф. зависимость

→ лог. зав.

→ упр. зав.

---→ транз. замыкание

$Dep(A, B)$

Этап III - Построение ярусно параллельной формы

1. A
2. B; C; D; E; F
3. F; G
4. H

Алгоритм построения ЯПФ:

Шаг 1. Заполнение первого яруса ЯПФ

В первый ярус помещаются $A \nexists B Dep(B, A)$

Шаг 2. Пусть k ярусов заполнено, если $\forall A \exists_{i \leq k} \exists_{j \leq n_k} A = A_{ij}$, то ЯПФ построен, иначе шаг 3.

Шаг 3. В $k + 1$ ярус помещаются такие вершины $A \quad Dep(B, A) \implies \exists_{i \leq k} \exists_{j \leq n_k} A_{ij} = B$, шаг 2.

В итоге на каждом ярусе окажутся исключительно независимые вершины, их операторы могут **выполняться одновременно**.

Этап IV - Построение параллельного алгоритма

Метод построения параллельного алгоритма: Пусть известно p - число задач и $1 \leq \mu \leq p$ - номер задачи

Шаг 1. Распределение вершин первого яруса ЯПФ между задачами

$\mu \sim A_{1,\mu}; A_{1,\mu+p}; A_{1,\mu+2p} \dots$

Шаг 2. Пусть k ярусов распределено

Если ЯПФ закончилось то алгоритм построен, иначе переходим на шаг 3.

Шаг 3. Распределение $k + 1$ яруса ЯПФ по задачам параллельного алгоритма

$\mu \sim c(A_{k+1,\mu}); A_{k+1,\mu}; c(A_{k+1,\mu+p}); A_{k+1,\mu+p}; c(A_{k+1,\mu+2p}); A_{k+1,\mu+2p} \dots$

Если функция $c(A)$ - истина, то мы исполняем операторы вершины сразу за функцией.

Если ложь, тогда операторы следующей вершины не выполняются, а проверяются значение функции следующей за этой вершины.

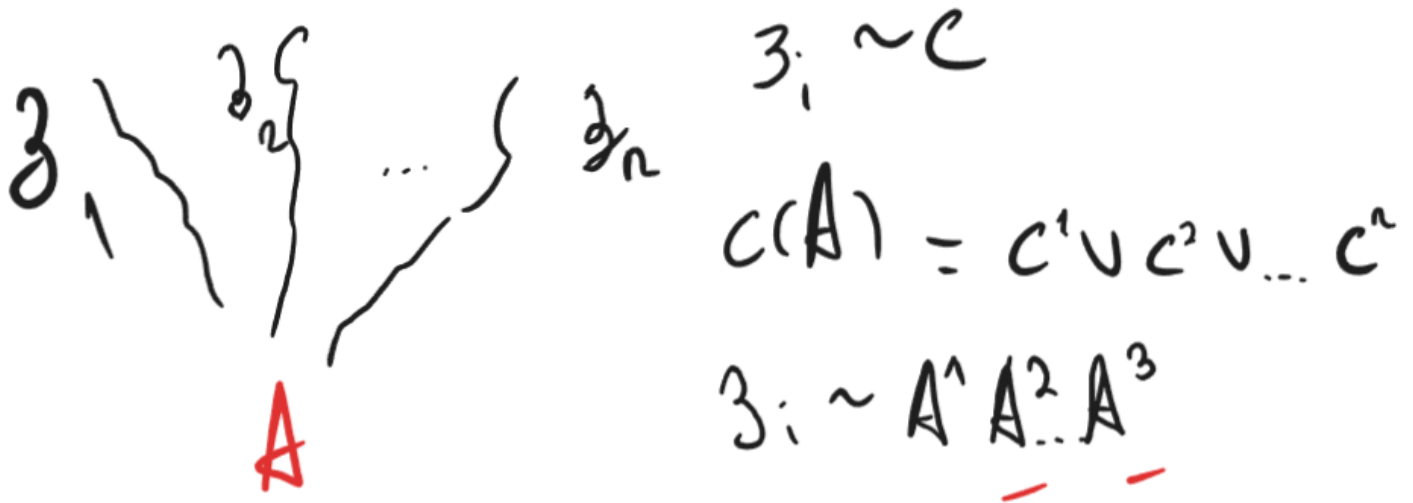
Если пока не определено, то вычислительный процесс прерывается, а μ — задача простаивает, пока значение не станет определенным.

Переход на шаг 2.

Формализм определения функции $c(A)$ Если далее говорится о пути, то имеется ввиду стандартный граф. Говоря о зависимости имеется ввиду граф зависимостей. Пусть через вершину A проходит n путей из входной вершины в выходную — $\xi^1, \xi^2, \xi^3, \dots, \xi^n$, тогда каждой ставим в соответствие $c^1, c^2, c^3, \dots, c^n$, принимающие значения истинно, если вычислительный процесс идущей по этому пути дошел до вершины A . Ложь, если заведомо известно, что пошел или пойдет другим путем, не содержащим A . c^i — неопределенно, если не одно из предыдущих утверждений пока не подходит. Тогда $c(A) = c^1 \cup c^2 \cup c^3 \dots \cup c^n$

Лекция 5 (07.03.26)

Выделим на данном пути, от который A зависит. Тогда $c(A_k)$, где $1 \leq k \leq K \implies c^i = c(A_1) \cap c(A_2) \cap \dots \cap c(A_K)$



$$(A_k) \begin{cases} c(A_k) \\ c(A_k = 1) - \text{Если } A_k - \text{распознаватель, и для реализации пути } \xi_i \text{ должно стать истинной} \\ c(A_k = 0) - \text{Если для реализации пути } \xi_i \text{ условие распознавателя } A_k - \text{ложно} \end{cases}$$

Параграф 2.2 Автоматическое распараллеливание циклических фрагментов последовательной программы

```
(*)
for i_1 = 1:n_1
  for i_2 = 1:n_2
    ...
    for i_k = 1:n_k
      T(i_1, i_2, ..., i_k)
    end
    ...
  end
end
```

$$I = \{(i_1, i_2, \dots, i_k) : 1 \leq i_g \leq n_g, 1 \leq g \leq k\}$$

I — пространство итераций

$(i_1, i_2, \dots, i_k)$ - вектор из пространства итераций

1. Отношение следования

Развернем конструкцию $(*)$ следующим образом:

$(\wedge) T(1,1, \dots, 1); T(1,1, \dots, 2); T(1,1, \dots, n_k); T(1,1, \dots, 2,1); \dots T(n_1, n_2, \dots, n_k)$

Теперь для ациклической конструкции строим стандартный граф. Ищем транзитивное замыкание отношение непосредственного следования, назовем его **отношения следования** на множестве вершин.

Между пространством итераций определенной $(*)$ и телами из развернутой ациклической конструкции $(\wedge)$, очевидно, существует взаимно однозначное отображение (биекция).

Тогда будем говорить, что вектор i из пространства итераций (связанный с телом T) следует за вектором i' (связанный с телом T'), если любая вершина из тела T конструкции $(\wedge)$ следует за любой вершиной из тела T' .

2. Отношение зависимость

По развернутой ациклической конструкции $(\wedge)$ строим граф зависимость.

Далее будем говорить, что вектор i (относящийся к T) зависит от i' (относящийся к T'), если хотя бы одна вершина из T зависит хотя бы от одной из.

Целью математической части лекции является поиск покрытия пространства итераций $\{I_q\}_{q=1, \overline{Q}}$

Если $I = \bigcup_{q=1}^Q I_q$, то $\{I_q\}_{q=1, \overline{Q}}$

При покрытии $I_1 \cap I_q = \emptyset$

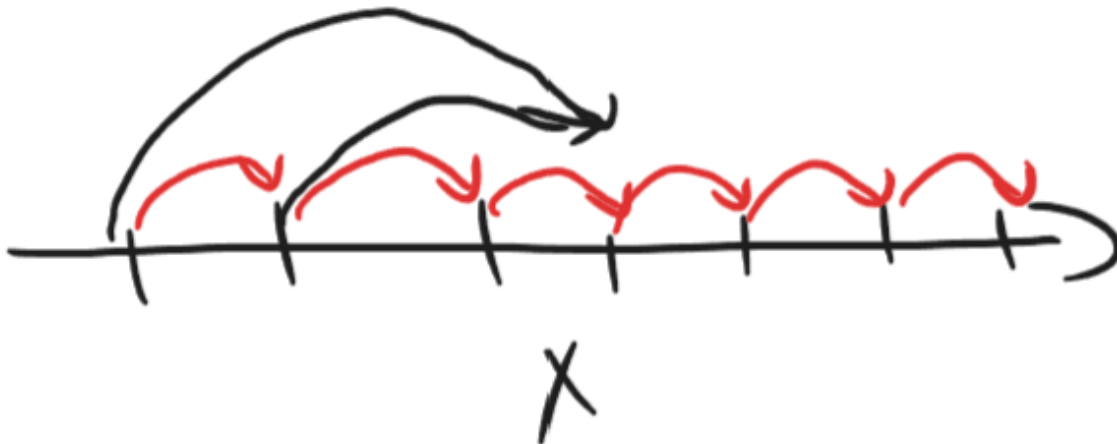
Наше цель найти покрытие, такое что любое его подмножество содержит исключительно независимые векторы.

В одном подмножестве покрытия итерации связанный с его векторами можно исполнять одновременно.

$$S = \frac{\prod_{j=1}^k n_j}{Q} = \frac{I_{\text{послед}}}{I_{\text{паралл}}} \rightarrow \text{Max} \quad - \quad \text{ускорение (speedup)}$$

Пример (?):

```
for i = 1:10
    x(i) = f(x(i-1))
end
```



Для дальнейшего построения параллельного алгоритма необходимо соблюдения следующих условий:

1. Параметры его тела внутри цикла не должны изменяться
2. В теле цикла отсутствует переход за циклическую конструкцию

3. Параметры цикла в теле цикла участвуют только в линейных операциях

Пункт 2.2.1 - Метод пирамид Шаг1. По циклической конструкции (*) строится пространство итераций на нём вводится отношения следования и зависимость.

На пространстве итераций выбираются вектора каждый из которых не зависит не от какого другого вектора этого пространства, назовем такие вектора **результатирующими**.

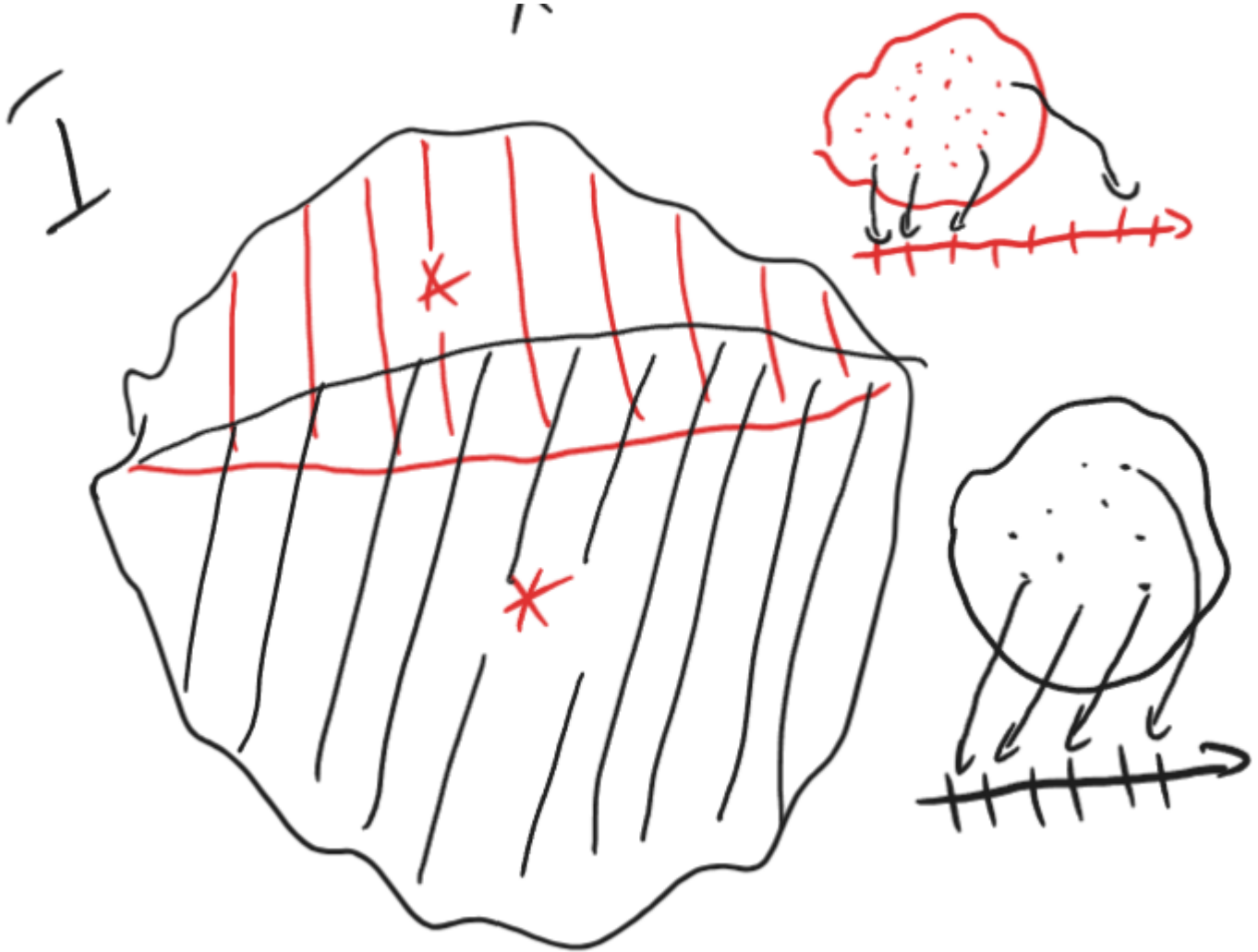
Шаг2. Каждому такому вектору поставим во взаимное однозначное соответствие (биекция) одну задачу параллельного алгоритма.

Кроме указанного вектора отнесем к ней также все вектора пространства итераций от которых данные **результатирующие** зависят.

Шаг3. Внутри каждой задачки упорядочим отнесенные к ней вектора в соответствие с отношением следования.

Алгоритм построен.

В отличие от других методов построения **Метод Пирамид** не характеризуется коммуникацией.



Лекция 6 (14.03.26)

Пример Метода пирамид

Последовательный алгоритм:

```

for i = 1:I
  for j = 1:J
    u(i,j) = f(u(i-1,j),u(i-1,j-1)mu(i-1,j+1))
  endfor
endfor

```

Параллельный алгоритм:

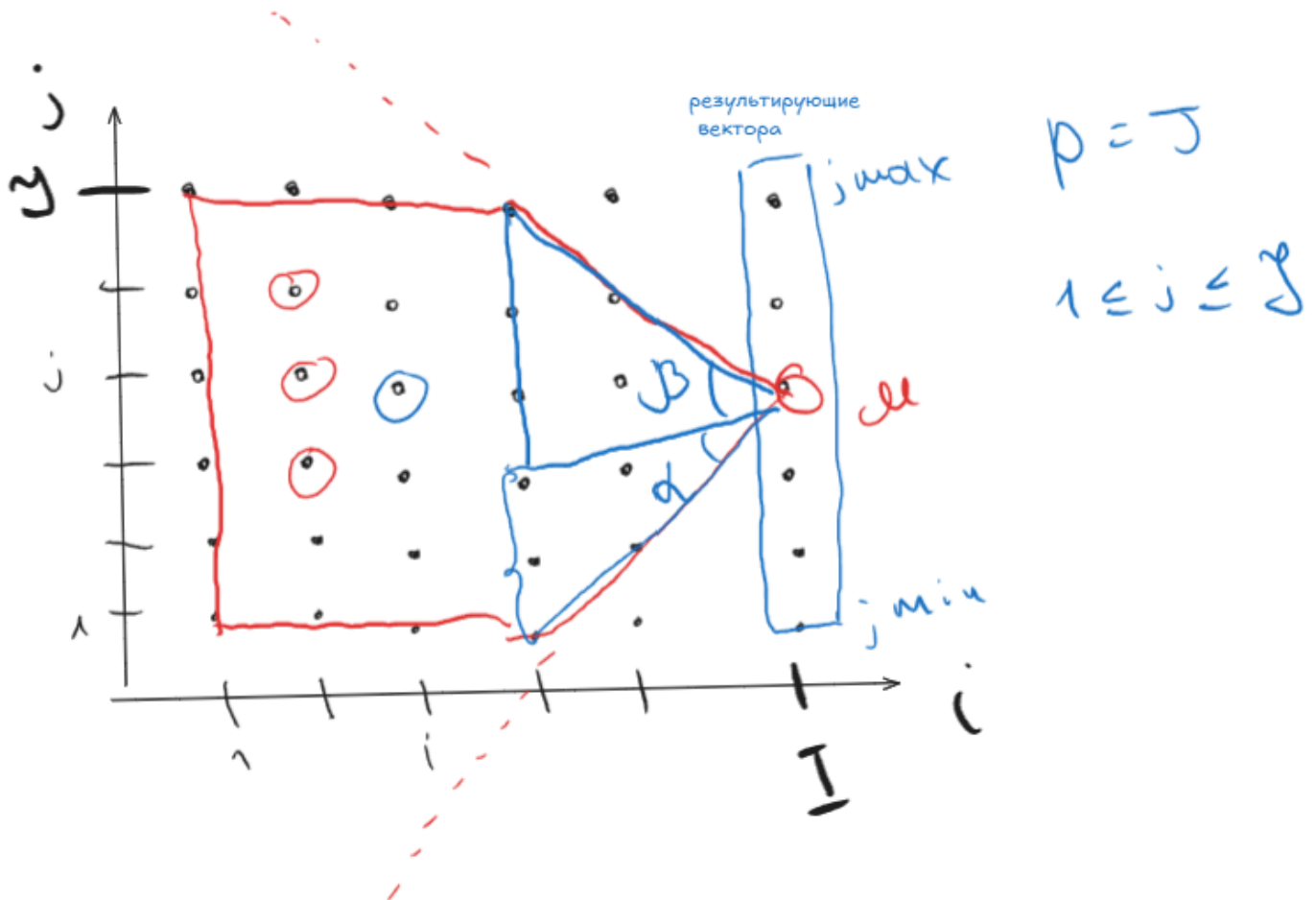
```

for i = 1:I
  for j = max{1,mu-(I-i)tg(alpha)} : min{J,mu+(I-i)tg(beta)}
    u(i,j) = f(u(i-1,j),u(i-1,j-1)mu(i-1,j+1))
  endfor
endfor

```

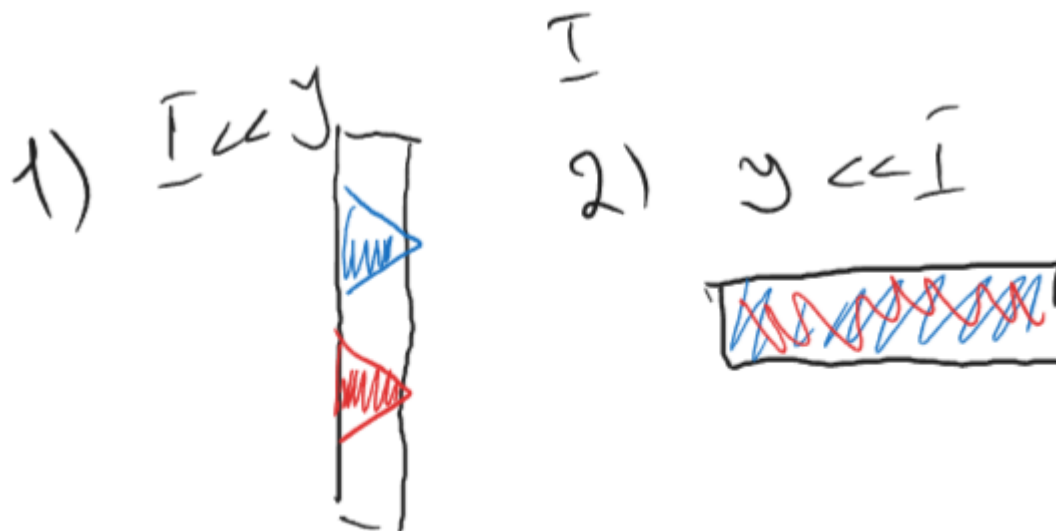
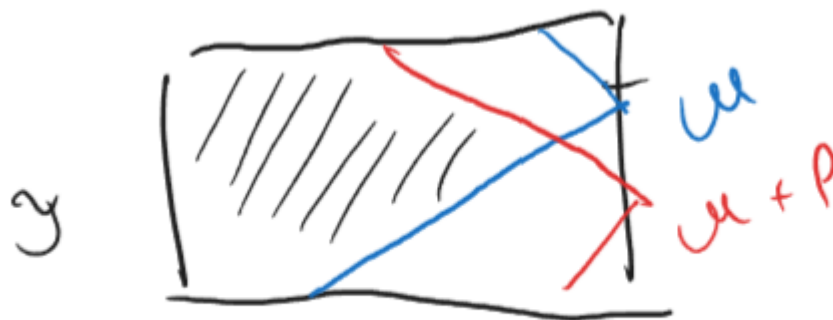
$\mu - (I - i)tg(\alpha) - j \min$

$\mu + (I - i)tg(\beta) - j \max$

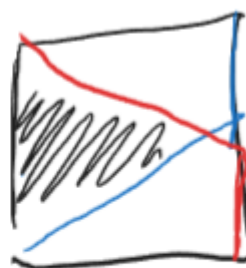
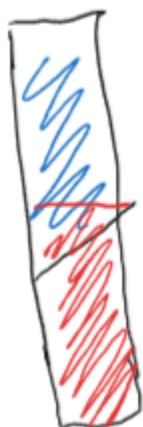


Замечания

1. Метод имеет смысл применять в 1) случае, а не 2):



2. На практике задачи с близкими номерами μ принято объединять



3. В отличие от остальных методов в данном допускается использование параметров цикла внутри цикла в нелинейных выражениях

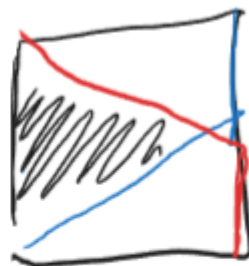
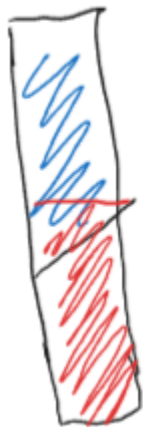
Пункт 2.2.2 Метод параллелепипедов

В отличие от всех остальных методов позволяет работать с простыми циклами

Последовательный алгоритм:

```
for i = 1:10
    x(i) = f(x(i-4))
endfor
```

Параллельный алгоритм (вариант 1):



$\mu = 1$

1

3

5

7

9

$\mu = 2$

2

4

6

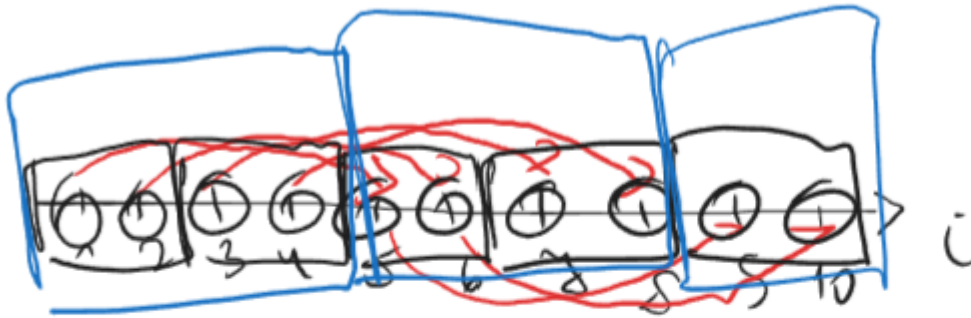
8

10

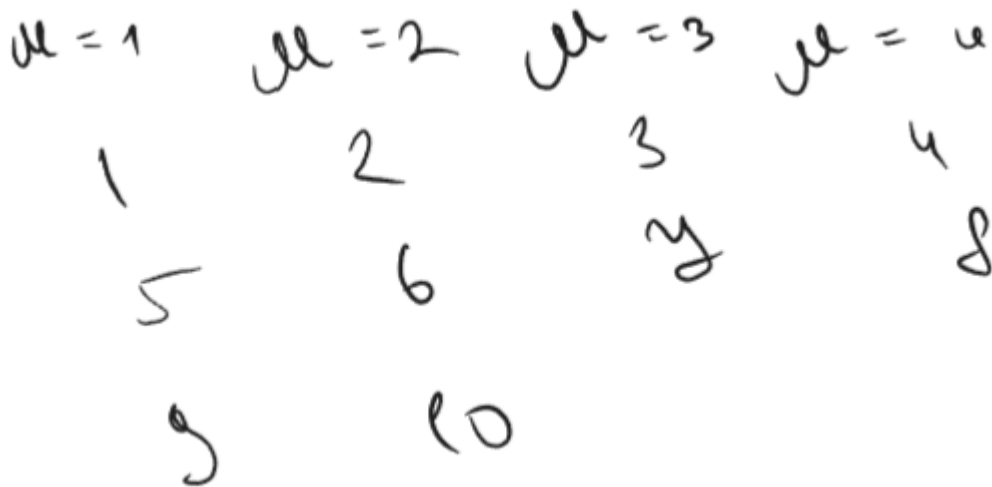
```
for i = mu:2:10
    x(i) = f(x(i-4))
endfor
```

$S = 2$

Параллельный алгоритм (вариант 2):



$$\rho = 2 \quad 1 \leq \mu \leq 2$$



```
for i = mu:4:10
    x(i) = f(x(i-4))
endfor
```

$$S = \frac{10}{3}$$

Шаг1. Строим пространство итераций, наносим на него отношение следования и зависимости.

Шаг2. На пространстве итераций ищем различные покрытия, каждый из которых представляет из себя набор k -мерных параллелепипедов.

```
for i_1 = 1:n_1
    for i_2 = 1:n_2
        ...
        for i_k = 1:n_k
            T(i_1, i_2, ..., i_k)
        endfor
    endfor
endfor
```

Шаг3. Для каждого из этих покрытий выбираем параллелепипед максимального объема и на его ребрах строим новую систему координат.

Количество задач параллельного алгоритма будет равным числу векторов в выбранном промежутке.

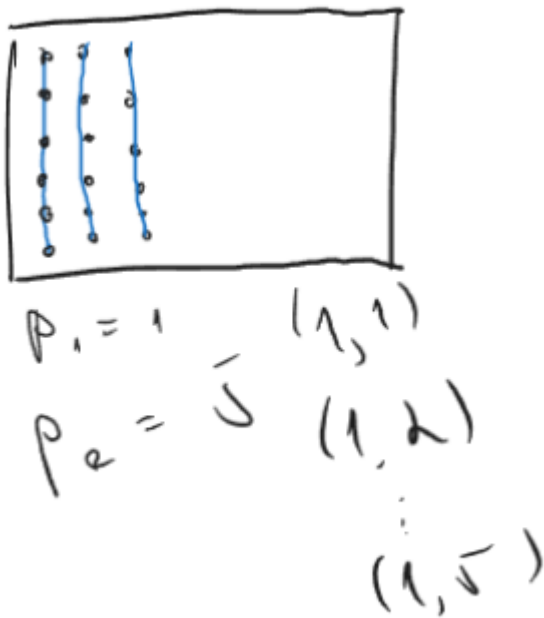
Если длины сторон параллелепипеда равны $p_1, p_2, \dots, p_k$ (где p_i есть разница координат последнего и первого вектора по i направлению + 1), то для прямоугольного параллелепипеда общее число задач есть: $p = \prod_{i=1}^k p_i$

За номер задачи примем координату вектора в выбранном параллелепипеде в новой система координат

$(g_1, g_2, \dots, g_k)$

Может случится, что размерность выбранного параллелепипеда окажется меньше пространства итераций, в этом случае одна или несколько p будут приняты за 1

А в номере задачи соответствующая ему координата также будет принята за единицу



Шаг4. Отнесем к ведению каждой задачи не более одного вектора из каждого подмножества выбранного покрытия

Внутри каждой задачи совокупность отнесенных к ней векторов расположены в соответствии с отношением следования, таким образом:

```
for i_1 = g_1:p_1:n_1
  for i_2 = g_2:p_2:n_2
    ...
    for i_k = g_k:p_k:n_k
      T(i_1,i_2,...,i_k)
      + коммуникация
    endfor
    ...
  endfor
endfor
```

Двумерный пример:

```
for i=1:7
  for j=1:3
    x(i,j) = f(x(i-2,j-1))
  endfor
endfor
```

Параллельный:

```

for i=g_1:p_1:7
  for j=g_2:p_2:3
    if (i>2) and (j>1) then
      recv(x(i-2,j-1), (g_1,g_2-1))

      x(i,j) = f(x(i-2,j-1))

      if(i<6) and (j<3) then
        send(x(i,j),(g_1,g_2+1))
      endif
    endif
  endfor
endfor

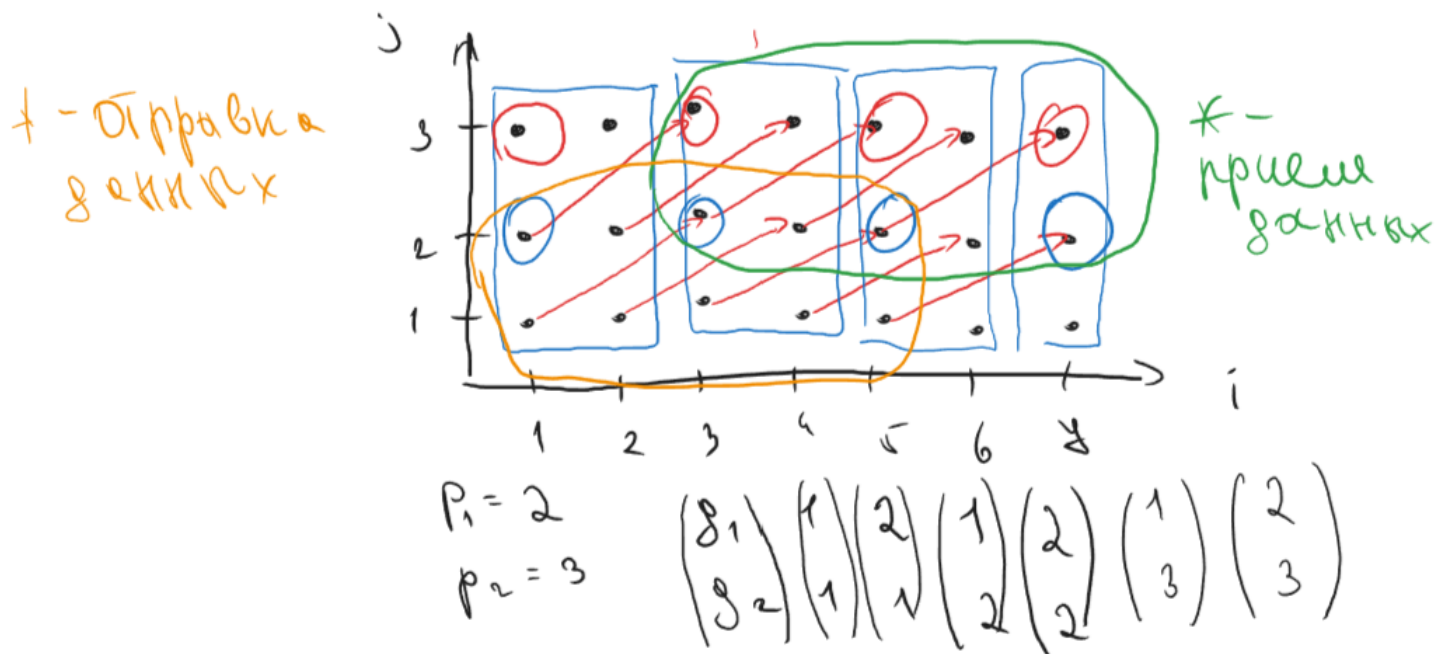
```

Для задачи в кружочке:

```

for i=1:2:7 ⇒ i = 1,3,5,7
  for j=3:3:3 ⇒ j = 3
    T(i,j)
  endfor
endfor

```



Как в прошлом случае количество задач найденных таким образом много больше числа используемых устройств, поэтому задачи объединяют.

Глава 3 Векторные алгоритмы вычислительной линейной алгебры

Лекция 7 (21.03.26)

Параграф 3.1 Векторные алгоритмы умножения матриц

Пункт 3.1.1 Основные векторные операции Операция с вектором (аппаратно)

1. $z = x + y$, где $z, x, y \in R$ - сложение
2. $z = \alpha x$, где $\alpha \in R$ - умножение вектора на скаляр
3. $a = x^T y$ - скалярное произведение

4. $z = x * y$ - покомпонентное произведение

5. $z = \alpha_x + y$ - saxpy (scalar α x plus y)

Операции с матрицами

1. $z = Ax + y$, где $A \in R^{n \times m}$ $z, y \in R^{n \times 1}$ $x \in R^{m \times 1}$ - gaxpy (gated A x plus y)

$$\begin{pmatrix} | \\ | \end{pmatrix} = \begin{pmatrix} | \\ | \end{pmatrix} \begin{pmatrix} | \\ | \end{pmatrix} + \begin{pmatrix} | \\ | \end{pmatrix}$$

$z \qquad \qquad A \qquad \qquad x \qquad \qquad y$



$z \leftarrow z + y$

(или с умножением)

gaxpy через скалярное произведение

```
for i = 1:n
    y(i) = A(i,:)x+y(i)
endfor
```

План Б. Выражение gaxpy через saxpy

$$z = \begin{pmatrix} | \\ | \\ | \end{pmatrix} = \begin{pmatrix} | \\ | \\ | \end{pmatrix} \begin{pmatrix} | \\ | \\ | \end{pmatrix} + \begin{pmatrix} | \\ | \\ | \end{pmatrix} = \begin{pmatrix} | \\ | \\ | \end{pmatrix} x^{(1)} + \begin{pmatrix} | \\ | \\ | \end{pmatrix} x^{(2)} + \dots + \begin{pmatrix} | \\ | \\ | \end{pmatrix} x^{(m)} + \begin{pmatrix} | \\ | \\ | \end{pmatrix}$$

z A x y $A(:,1)$ $A(:,2)$ $A(:,m)$ y

```
for j = 1:m
    y = y + A(:,j)x(j)
endfor
```

2. $A = A + xy^T$, где $A \in R^{n \times m}$ $x \in R^{n \times 1}$ $y \in R^{m \times 1}$

Вся операция вместе - называется модификация матрицы внешним произведением

```
for i=1:n
    A(i,:) = A(i,:) + x(i)y^T - строчное saxpy
endfor

for j=1:m
    A(:,j) = A(:,j) + xy(j) - столбцовое saxpy
endfor
```

Пункт 3.1.2 Векторные алгоритмы умножения матриц Умножение по определению:

```
for i=1:n
    for j=1:n
        for k=1:n
            c(i,j) = A(i,k)B(k,j)
        endfor
    endfor
endfor
```

Цикл k преобразуем в скалярное произведение:

```
for i=1:n
    for j=1:n
        c(i,j) = A(i,:)B(:,j)
    endfor
endfor
```

Цикл j убираем, получаем saxpy:

```
for i=1:n
    c(i,:) = A(i,:)B - строчное saxpy
endfor
```

$z = xA + y$, где $A \in R^{n \times m}$ $x \in R^{n \times 1}$ $y \in R^{m \times 1}$ - строчное saxpy

Название алгоритма	Векторная операция	Название операции	$c = A * B$ Доступ к пам
ijk	скалярное произведение	строчное saxpy	смещенная
kij	строчное saxpy		по строкам

```

for k=1:n
    for i=1:n
        for j=1:n
            c(i,j) = c(i,j) + A(i,k)B(k,j)
        endfor
    endfor
endfor

```

Свернем j цикл (сахру строчное):

```

for k=1:n
    for i=1:n
        c(i,:) = c(i,:) + A(i,k)B(k,:) - сахру строчное
    endfor
endfor

```

Свернем i цикл (модификация матрицы):

```

for k=1:n
    c = c + A(:,k)B(k,:) - модификация матрицы
endfor

```

Блочный алгоритм:



A, B, C $d = \frac{n}{N} \leftarrow$ средняя
размер.



```
for i_b = 1:N
    i=1(i_b-1)alpha + 1 : i_b alpha
    for j_b = 1:N
        j=1(j_b-1)alpha + 1 : j_b alpha
        for k_b = 1:N
            k=1(k_b-1)alpha + 1 : k_b alpha
            c(i,j) = c(i,j) + A(i,k)B(k,j)
        endfor
    endfor
endfor
```

Параграф 3.2 Векторные алгоритмы решения СЛАУ

Пункт 3.2.1 Решение СЛАУ с матрицами треугольного вида $Lx = b$

$$\begin{pmatrix} l_{11} & 0 & 0 & \cdots & 0 \\ l_{21} & l_{22} & 0 & \cdots & 0 \\ l_{31} & l_{32} & l_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \cdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \cdots \\ b_n \end{pmatrix}$$

$$x_1 = \frac{b_1}{l_{11}}$$

$$x_2 = \frac{b_2 - l_{21}x_1}{l_{22}}$$

$$x_3 = \frac{b_3 - l_{31}x_1 - l_{32}x_2}{l_{33}}$$

$$x_i = \left(b_i - \sum_{j=1}^{i-1} \frac{l_{ij}x_j}{l_{ii}} \right)$$

$L(i, 1 : i - 1)(1 : i - 1)$ - скалярное произведение

```
for i=2:n
    b(i) = b(i) - L(i,1:i-1)b(1:i-1) / L(i,i)
endfor
```

Пусть L нижняя унитреугольная матриц

```
for i=2:n
    b(i) = b(i) - L(i,1:i-1)b(1:i-1) - строка
endfor
```

$$\begin{pmatrix} l_{11} & 0 & 0 & \cdots & 0 \\ l_{21} & l_{22} & 0 & \cdots & 0 \\ l_{31} & l_{32} & l_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \cdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \cdots \\ b_n \end{pmatrix} - \begin{pmatrix} l_{21} \\ l_{31} \\ l_{41} \\ \cdots \\ l_{n1} \end{pmatrix} - b'$$

$$x_1 = \frac{b_1}{l_{11}}$$

$$x_2 = \frac{b'_2}{l_{22}}$$

```
for j=1:n-1
    b(j) = b(j)/L(j,j)
    b(j+1:n) = b(j+1:n) - L(j+1:n,j)b(j) - saxpy (столбец)
endfor
b(n)=b(n)/L(n,n)
```

Пусть L нижняя унитреугольная матриц

```
for j=1:n-1
    b(j+1:n) = b(j+1:n) - L(j+1:n,j)b(j) - saxpy (столбец)
endfor
```

$L = X = B$, где $L \in R^{n \times n}$ $X, B \in R^{n \times m}$

$$\frac{n}{N} = \alpha$$

$$L_{i_b, j_b} \in R^{\alpha \times \alpha}$$

$$B_{i_b}, X_{i_b} \in R^{\alpha \times m}$$

$$1 \leq i_b, j_b$$

$$n \begin{pmatrix} & & \\ & L_{i_b j_b} & \\ & & \end{pmatrix} \begin{pmatrix} & \\ X_{i_b} & \\ & \end{pmatrix}_m = \begin{pmatrix} & \\ B_{i_b} & \\ & \end{pmatrix}_m n$$

```

for j_b = 1:N
    L_{j_b, j_b} X_{j_b} = B_{j_b} - СЛАУ
    for i_b = j_b+1:N
        B_{i_b} = B_{i_b} - L_{i_b, j_b} X_{i_b}
    endfor
endfor
L_{N, N} X_N = B_N

```

Лекция 8 (28.03.26)

Пункт 3.2.2 Алгоритм LU разложения через модификацию внешним произведением

$$\begin{pmatrix} 1 & 0 \\ -\frac{x_2}{x_1} & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 \\ -\frac{x_2}{x_1} & 1 \end{pmatrix} - \text{матрица Гаусса}$$

$$\begin{pmatrix} 1 & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & 1 & 0 & \dots & 0 \\ 0 & 0 & \dots & -\frac{x_{k+1}}{x_k} & 1 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & -\frac{x_n}{x_k} & 0 & \dots & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_k \\ x_{k+1} \\ \dots \\ x_n \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_k \\ 0 \\ \dots \\ 0 \end{pmatrix}$$

$$\tau^{(k)} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ \frac{x_{k+1}}{x_k} \\ \vdots \\ \frac{x_n}{x_k} \end{pmatrix}$$

$$\tau_i^{(k)} = \begin{cases} 0, & \text{при } i \leq k \\ \frac{x_i}{x_k}, & \text{при } i > k \end{cases}$$

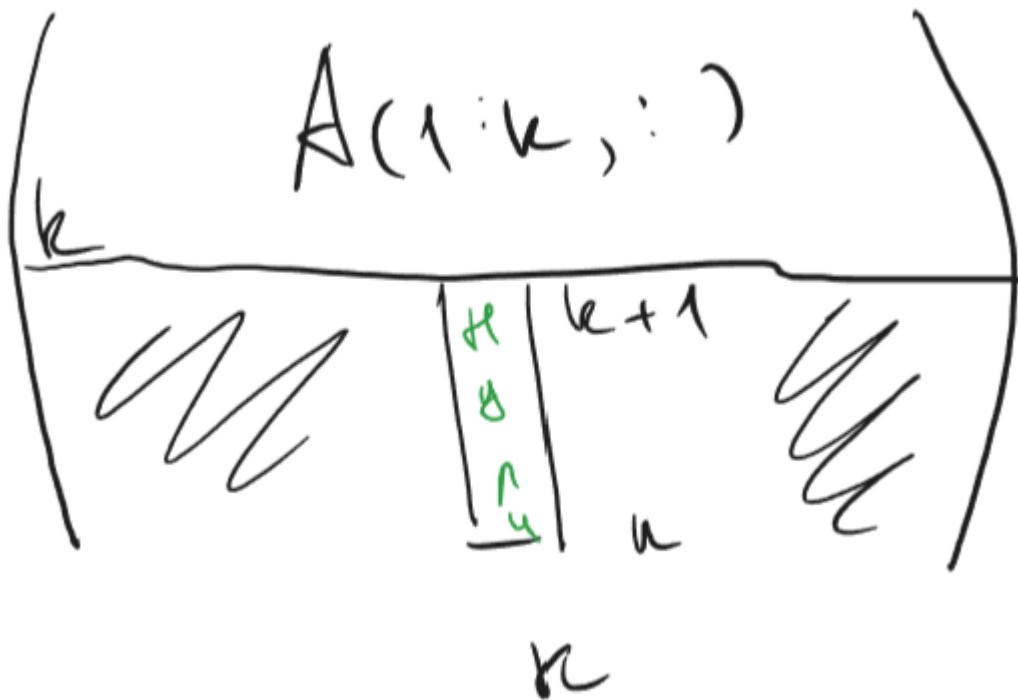
$$l_k = \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

$$\tau^{(k)} l_k^T = ???$$

$$\tau_i^{(k)} = \frac{a_{ik}}{a_{kk}}$$

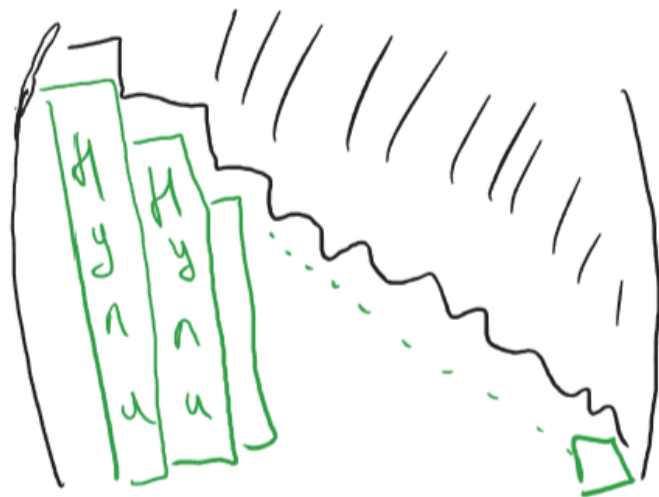
$I - \tau^{(k)} l_k^T$ — Мк матрица преобразования Гаусса

$$M_k A = (I - \tau^{(k)} l_k^T) A = A - \tau^{(k)} l_k^T A = A - \tau^{(k)} A(k, :)$$



$c = IAB$, где A — I на N , B — N на J , C — I на J

$$M_{n-1} \dots M_2 M_1 A$$

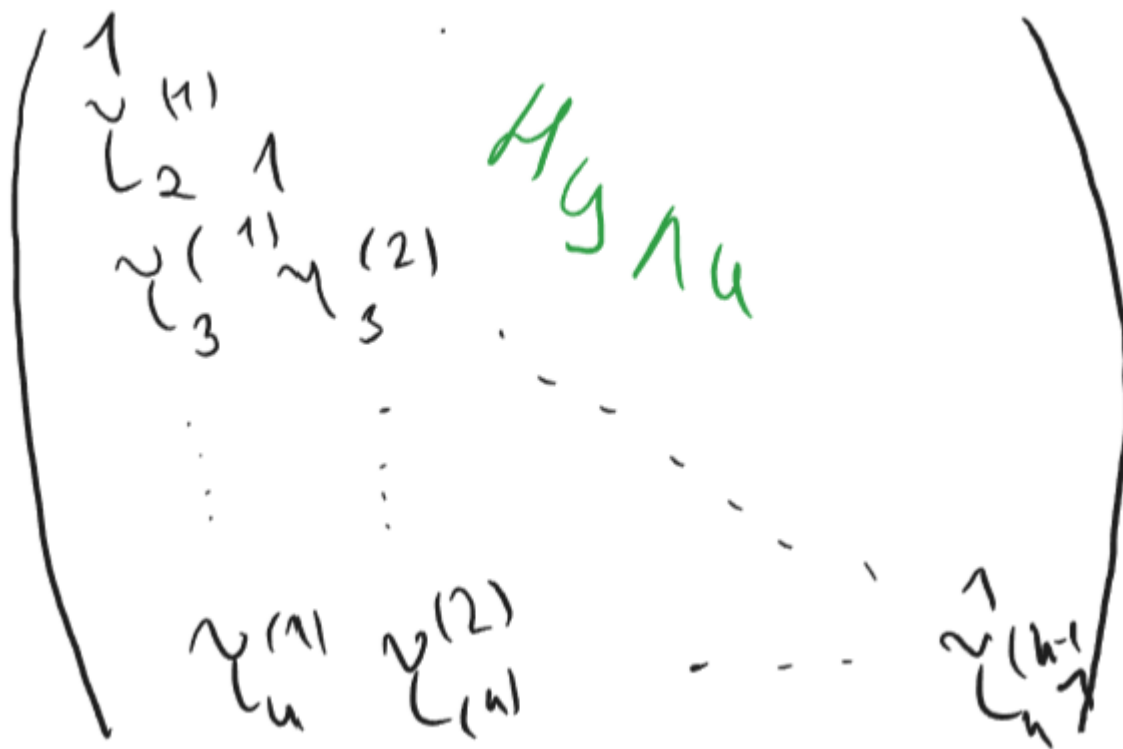


$$M_{n-1} \dots M_2 M_1 A = U$$

$$M_{n-2} \dots M_1 A = M_{n-1}^{-1} U$$

$$A = M_1^{-1} M_2^{-1} \dots M_{n-1}^{-1} U$$

$$L = \prod_{k=1}^{n-1} M_k^{-1} = I + \sum_{k=1}^{n-1} \tau^{(k)} l_k^T$$



Алгоритм LU разложения:

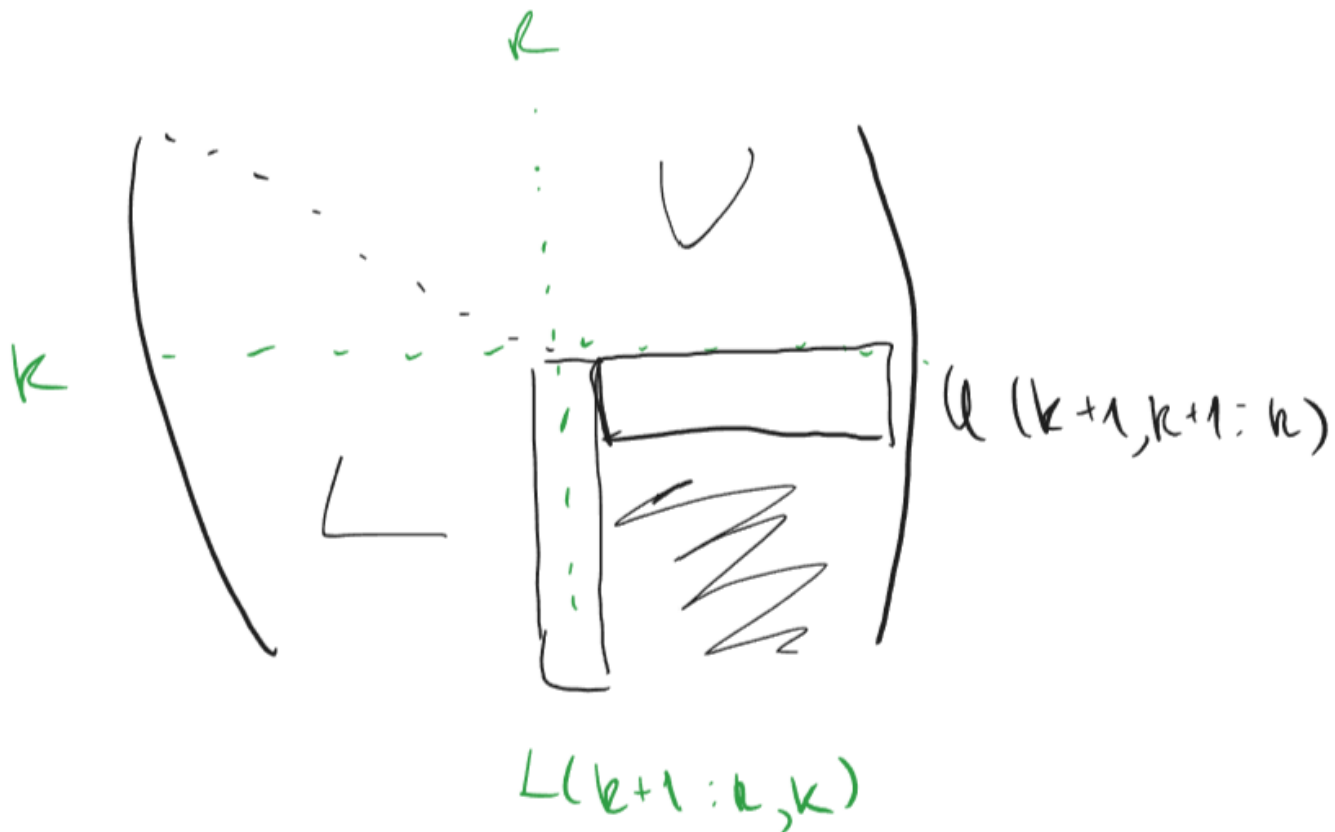
for k 1:n-1

$A(k+1:n,k) = A(k+1:n,k)A(k,k)$ - k-ый вектор множителя гаусса (k-ый столбец матрицы l)

$A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)A(k,k+1:n)$

endfor

На k -ом шаге алгоритма находи $k + 1$ -ый столбец:



	Название	Вектор	Матричные операции	Доступ к данным
C	kji	Столбцовый saxpy	Модификация матрицы внешним произведением	Столбцовый
C	kij	Строчный saxpy	Модификация матрицы внешним произведением	Строчный
Fortran	jik	Скалярное произведение	Столбцовое saxpy	Смешанное
Fortran	jki	Столбцовое saxpy	Столбцовое saxpy	Столбцовый

При возможности реализовать одно и тоже через модификацию матрицы или saxpy предпочтение отдают saxpy, потому что по сравнению с модификацией матрицей saxpy характеризуется вдвое меньшим объемом коммуникаций между кэш памятью процессора и регистрами

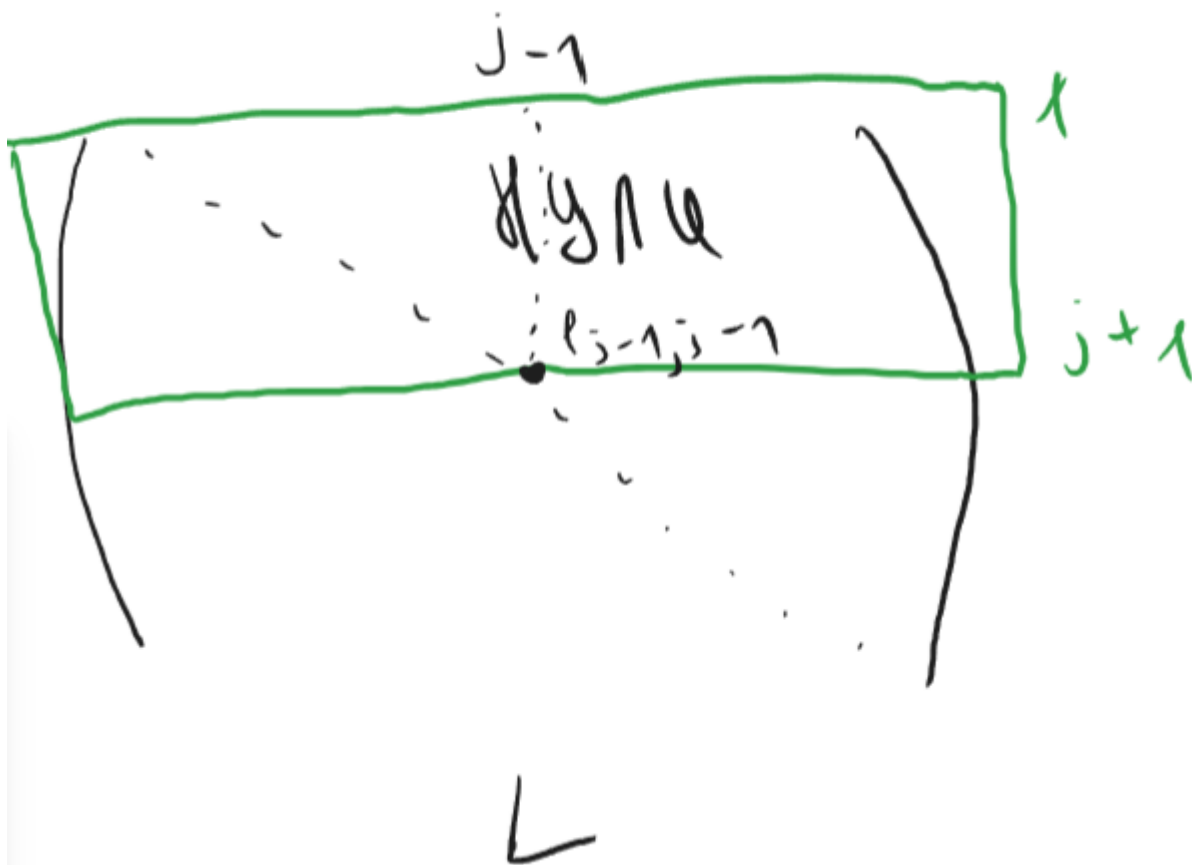
Пункт 3.2.3 Производственное LU разложение через saxpy

$$A = LU$$

Положим что к j -ому шагу алгоритма первые $j - 1$ столбцов L и U уже найдены, найдем j

$$A(:, j) = LU(:, j)$$

$$1) A(1 : j - 1, j) = L(1 : j - 1, :)U(:, j)$$



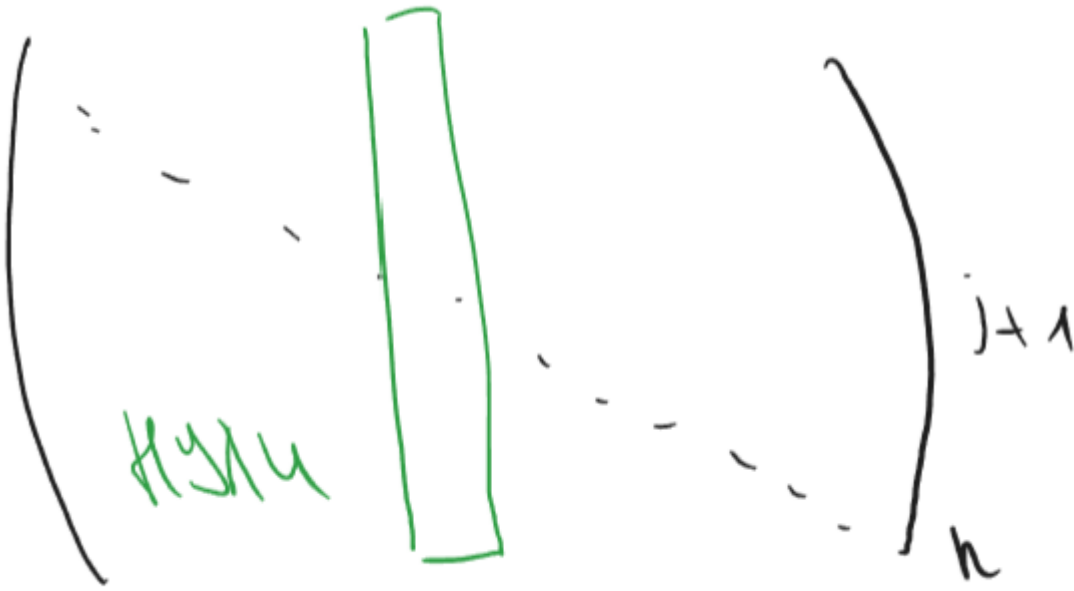
$A(1:j-1, j) = L(1:j-1, 1:j-1)U(1:j-1, j)$, получаем:

$L(1:j-1, 1:j-1)$ - знаем

$U(1:j-1, j)$ - ищем

Это СЛАУ $Lx = b$, где $b = A(1:j-1, j)$, $x = U(1:j-1, j)$

$$2) A(j:n, j) = L(j:n, :)U(:, j)$$



$$A(j : n, j) = L(j : n, 1 : j)U(1 : j, j)$$

$$A(j : n, j) = \sum_{k=1}^j L(j : n, k)U(k, j)$$

$$A(j : n, j) = \sum_{k=1}^{j-1} L(j : n, k)U(k, j) + L(j : n, j)U(j, j)$$

$$A(j : n, j) = L(j : n, 1 : j-1)U(1 : j-1, j) + L(j : n, j)U(j, j), \text{ получаем:}$$

$$A(j : n, j) - \text{знаем}$$

$$L(j : n, 1 : j-1) - \text{знаем}$$

$$U(1 : j-1, j) - \text{знаем}$$

$$L(j : n, j) - \text{неизвестно } V(j : n)$$

$$V(j : n) = A(j : n, j) - L(j : n, 1 : j-1)U(1 : j-1, j) - \text{даем}$$

$$V(j : n) = L(j : n, 1 : j-1)U(j, j)$$

$$\begin{pmatrix} \square \\ & \square \\ & & \square \end{pmatrix} = \begin{pmatrix} \square \\ & \square \\ & & 1 \end{pmatrix} \begin{pmatrix} \square \\ & \square \\ & & \square \end{pmatrix}$$

$$V(j) = L(j:j) V(j,j)$$

$$L(j+1:n, j) = V(j+1:n) U(j, j)$$

```

for j=1:n
    1) решаем СЛАУ  $L(1:j-1, 1:j-1)U(1:j-1, j) = A(1:j-1, j)$ 
    2) гаппу  $V(j:n) = A(j:n, j) - L(j:n, j-1)U(1:j-1, j)$ 
     $U(j, j) = V(j)$ 
     $L(j+1:n, j) = V(j+1:n)U(j, j)$ 
endfor

```

```

for j=1:n
    for k=1:j-2
         $A(k+1:j-1, k) = A(k+1:j-1, k) - A(k+1:j-1, k)A(k, j)$ 
    endfor
     $A(j:n, j) = A(j:n, 1:j-1)A(1:j-1, j) \Rightarrow$  вниз смотри 1), 2)
     $A(j+1:n, j) = A(j+1:n, j)A(j, j)$ 
endfor

```

1)

```

for k=1:j-1
     $A(j:n, j) = A(j:n, j) - A(j:n, k)A(k, j)$ 
endfor

```

2)

```

for i=j:n
     $A(i, j) = A(i, j) - A(i, 1:j-1)A(1:j-1, j)$  - скалярное произведение
endfor

```

Лекция 9 (04.04.2026)

Пункт 3.2.4 Блочный алгоритм LU разложения

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

$$r \ll n$$

$$A \in \mathbb{R}^{n \times n}$$

$$A_{11}, L_{11}, I_r, U_{11} \in \mathbb{R}^{n \times r}$$

$$A_{12}, U_{12} \in \mathbb{R}^{r \times (n-r)}$$

$$A_{21}, L_{21} \in \mathbb{R}^{(n-r) \times n}$$

$$A_{22}, I_{n-r} \in \mathbb{R}^{(n-r) \times (n-r)}$$

$$\begin{aligned} \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} &= \\ &= \begin{pmatrix} L_{11} & \text{нули } ((n-r) \text{ на } r) \\ L_{21} & I_{n-r} \end{pmatrix} \times \begin{pmatrix} I_r & \text{нули} \\ \text{нули } ((n-r) \text{ на } r) & \tilde{A} \end{pmatrix} \times \begin{pmatrix} U_{11} & U_{12} \\ \text{нули} & I_{n-r} \end{pmatrix} = \\ &= \begin{pmatrix} L_{11}U_{11} & L_{11}U_{12} \\ L_{21}U_{11} & \tilde{A} + L_{21}U_{12} \end{pmatrix} \end{aligned}$$

Алгоритм I:

$$1) A_{11} = L_{11}U_{11}$$

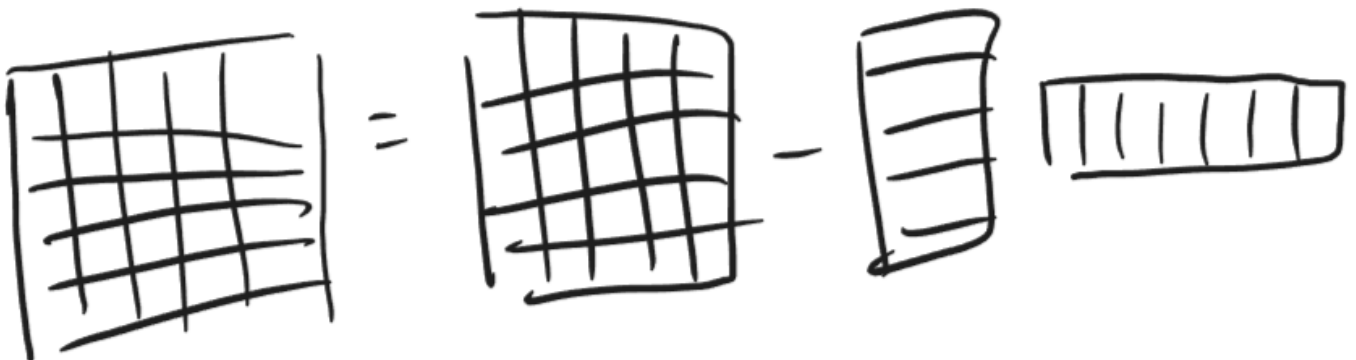
Производим LU разложение матрицы A по любому векторному алгоритму

$$2) A_{12} = L_{11}U_{12} \rightarrow \text{слау } LX = B$$



$$3) A_{21} = L_{21}U_{11} \rightarrow \text{слау } XU = B$$

$$4) A_{22} = \tilde{A} + L_{21}U_{12} \Rightarrow \tilde{A} = A_{22} - L_{21}U_{12}$$



5) Примем $A = \tilde{A}$, пока матрица не кончится

Алгоритм II (Fortran алгоритм):

1) Производим LU разложение матрицы A через гауру **столбцовое**, но не полностью, а r первых шагов

U_{11}, L_{11}, L_{21}

2) — (бывший 2))

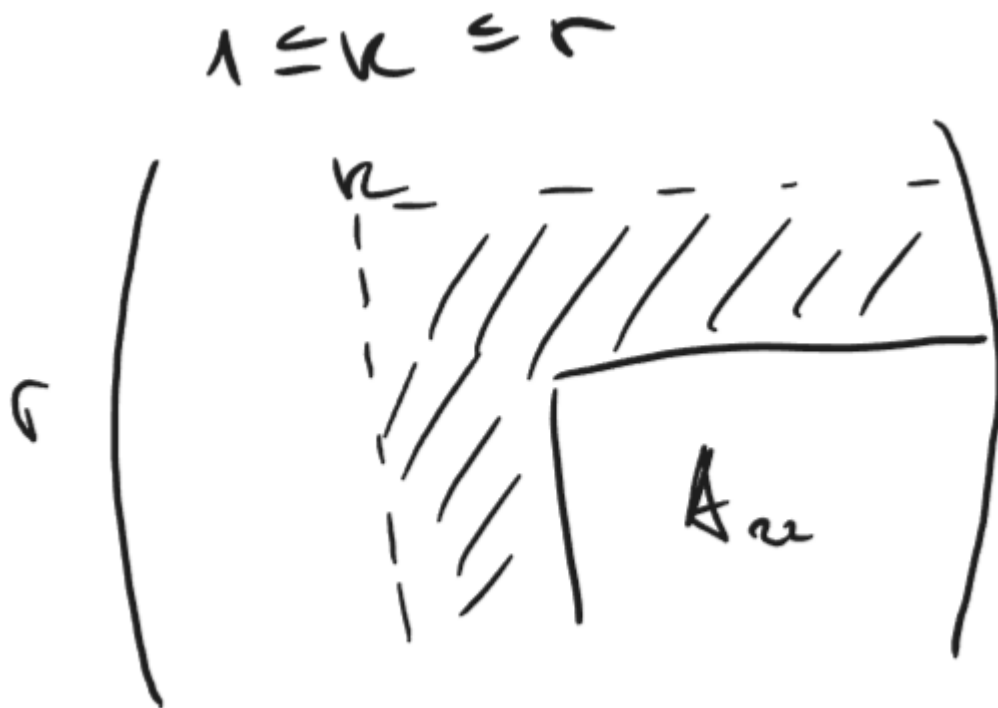
3) — (бывший 4))

4) — (бывший 5))

Алгоритм III:

1) Производим r первых шагов LU разложения матрицы A , через модификацию матрицы внешним произведением
(не затрагивая блок A_{22})

$L_{11}, U_{11}, L_{21}, U_{12}$



2) — (бывший 4))

3) — (бывший 5))

Алгоритм IV (C алгоритм):

1) Производим r шагов LU разложения матрицы A , но через гауру **строчное**

r строк L и U , L_{11}, U_{11}, U_{12}

2) — (бывший 3))

3) — (бывший 4))

4) — (бывший 5))



Параграф 3.3 Разложение Холецкого

$$A = LU$$

$$A = L \cdot L^T$$

$$A = G \cdot G^T$$

Пункт 3.3.1 Разложение Холецкого через модификацию матрицы внешним произведением Пусть $r = 1$

$$A = \begin{pmatrix} \alpha & v^T \\ v & B \end{pmatrix} =$$

$$= \begin{pmatrix} \beta & \text{нули} \\ \frac{v}{\beta} & I_{n-1} \end{pmatrix} \begin{pmatrix} 1 & \text{нули} \\ \text{нули} & B - \frac{v \cdot v^T}{\alpha} \end{pmatrix} \begin{pmatrix} \beta & \frac{v^T}{\beta} \\ \frac{v}{\beta} & I_{n-1} \end{pmatrix}$$

$$\alpha \in \mathbb{R}$$

$$\alpha = a_{11}; \beta = \sqrt{\alpha}$$

$$v = A(2:n, 1)$$

$$v \in \mathbb{R}^{(n-1) \times 1}$$

$$B = A(2:n, 2:n)$$

$$B \in \mathbb{R}^{(n-1) \times (n-1)}$$

$$B - \frac{v \cdot v^T}{\alpha} \text{ — модификация матрицы внешним произведением}$$

$$B - \frac{v \cdot v^T}{\alpha} = G_1 \cdot G_1^T = \begin{pmatrix} \beta & \text{нули} \\ \frac{v}{\beta} & G_1 \end{pmatrix} \begin{pmatrix} \beta & \frac{v^T}{\beta} \\ \text{нули} & G_1^T \end{pmatrix}$$

$$G_1 \in \mathbb{R}^{(n-1) \times (n-1)}$$

$$G = \begin{pmatrix} \beta & \text{нули} \\ \frac{v}{\beta} & G_1 \end{pmatrix}$$

Матричный вид:

```
for k=1:n
    A(k:n,k) = A(k:n,k)/(sqrt(A(k,k)))
    A(k+1:n,k+1:n) = A(k+1:n,k+1:n) - A(k+1:n,k)[A(k+1:n,k)]^T
endfor
```

Векторный вид:

```
for k=1:n
    A(k:n,k) = A(k:n,k)/sqrt(A(k,k))
    for j=k+1:n
        A(j:n,j) = A(j:n,j) - A(j:n,k)A(j,k)
    endfor
endfor
```

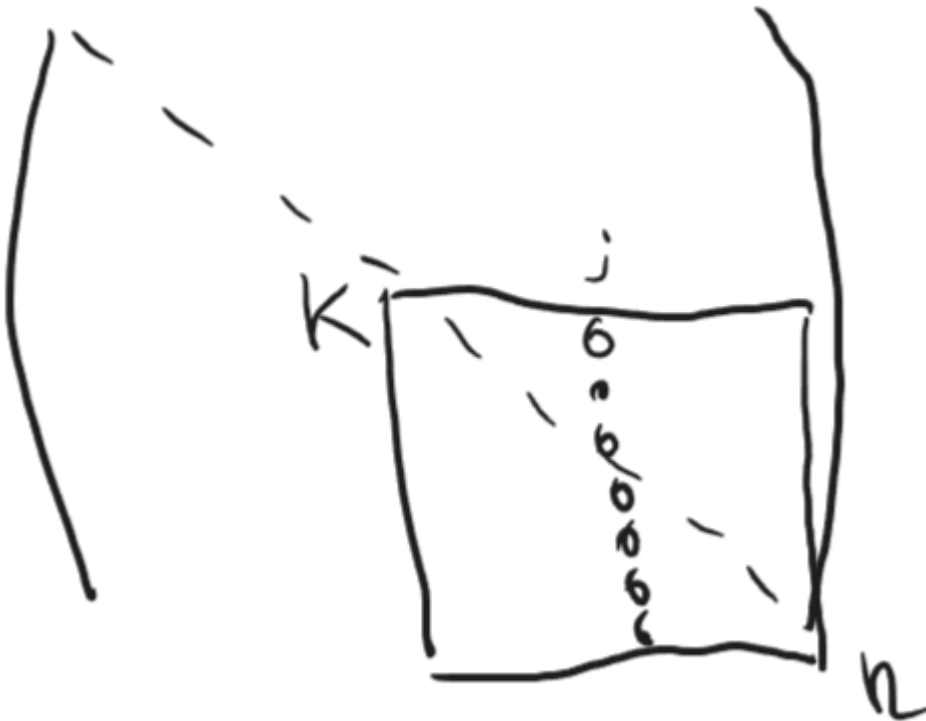


Таблица для Холецкого:

Название алгоритма	Векторная операция	Матричная операция	Доступ к памяти
kji	столбцовый saxpy	модификацию матрицы внешним произведением	по столбцам
kij	строчный saxpy	модификацию матрицы внешним произведением	по строкам

Лекция 10 (11.04.26)

Пункт 3.3.2 Разложение Холецкого через гахру

Пусть к j -ому шагу алгоритма первые $j - 1$ -ый столбцов матрицы J найдены, отыщем j -ий.

$$I \begin{pmatrix} \text{ } \\ c \end{pmatrix} = \begin{pmatrix} I & A \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix}$$

$$A = G * G^T \quad A(:, j) = G * G^T(:, j)$$

$$1) \quad A(1:j-1, j) = G(1:j-1, j) G^T(:, j) \quad A(1:j-1, j) = G(1:j-1, 1:j-1) G^T(1:j-1, j)$$

$$2) \quad A(j:n, j) = G(j:n, j) G^T(:, j)$$

$$A(j:n, j) = G(j:n, 1:j) G^T(1:j, j)$$

$$A(j:n, j) = \sum_{k=1}^j G(j:n, k) G^T(k, j)$$

$$A(j:n, j) = \sum_{k=1}^{j-1} G(j:n, k) G^T(k, j) + G(j:n, j) G^T(j, j)$$

$$A(j:n, j) = G(j:n, 1:j-1) G^T(1:j-1, j) + G(j:n, j) G^T(j, j)$$

$$G(j:n, j) G^T(j, j) \quad (\text{не знаем, назовём} - V(j:n))$$

$$V(j:n) = A(j:n, j) - G(j:n, 1:j-1) G^T(1:j-1, j)$$

$$V(j:n) = G(j:n, j) G^T(j, j)$$

$$V(j:n) = G(j, j) G(j, j)$$

$$G(j, j) = \sqrt{V(j)}$$

$$G(j+1:n, j) = V(j+1:n) G(j, j)$$

Алгоритм:

```
for j=1:n
    A(j:n,j) = A(j:n,j) - A(j:n,1:j-1) * [A(j,1:j-1)]^T #столбцовый гахру
    #V(j,n)    #A(j:n,j)    #G(j:n,1:j-1)    #G^T(1:j-1,j)

    A(j:n,j) = A(j:n,j) / sqrt(A(j,j))
endfor
```

преобразуем операцию гахру через скалярное произведение (Алгоритм jik) $\Rightarrow$:

```
for i=j:n
    A(i,j) = A(i,j) - A(i,1:j-1) * [A(j,1:j-1)]^T
endfor
```

преобразуем операцию гахру через столбцовое сахру (Алгоритм jki) $\Rightarrow$:

```

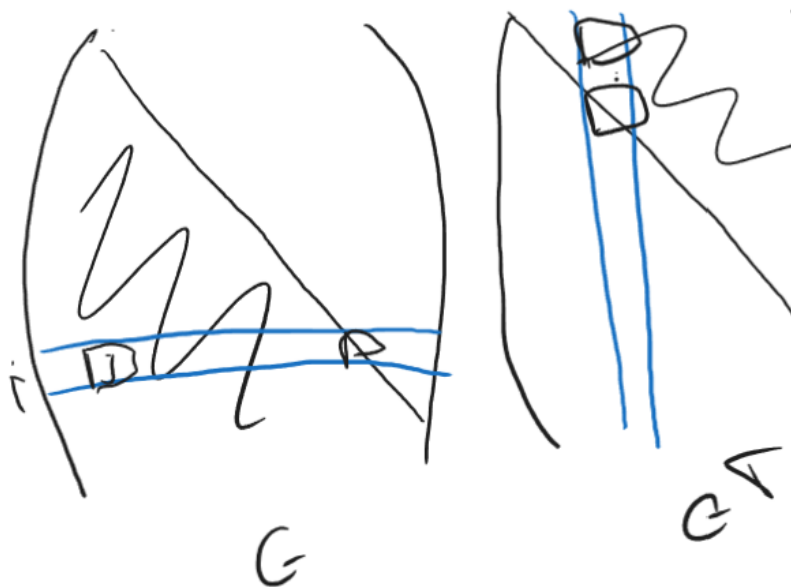
for k=1:j-1
    A(j:n,j) = A(j:n,j) - A(j:n,k) A(j,k)
endfor

```

	Название алгоритма	Векторная операция	Матричная операция	Доступ к памяти
C	jik	Скалярное произведение	Столбцовое дахру	Строчный
C/Fortran	jki	Столбцовое дахру	Столбцовое дахру	Столбцовый

Пункт 3.3.3 Блочные алгоритмы метода Холецкого

Юрий Магнел



Пункт I Блочный алгоритм Холецкого на основе ???

$$G_{ij} \in \mathbb{R}^{\alpha \times \alpha}$$

$$1 \leq i, j \leq N; \quad N = \frac{n}{\alpha}$$

$$A_{ij} = \sum_{k=1}^N G_{ik} G_{kj}^T = \sum_{k=1}^j G_{ik} G_{kj}^T$$

$$A_{ij} = \sum_{k=1}^{j-1} G_{ik} G_{kj}^T + G_{ij} G_{jj}^T$$

$$S_{ij} = A_{ij} - \sum_{k=1}^{j-1} G_{ik} G_{kj}^T$$

При поиске блока G_{ij} будем обходить матрицу G в первом цикле слева направо по блочным столбцам, во втором цикле сверху вниз по блокам выделенного столбца.

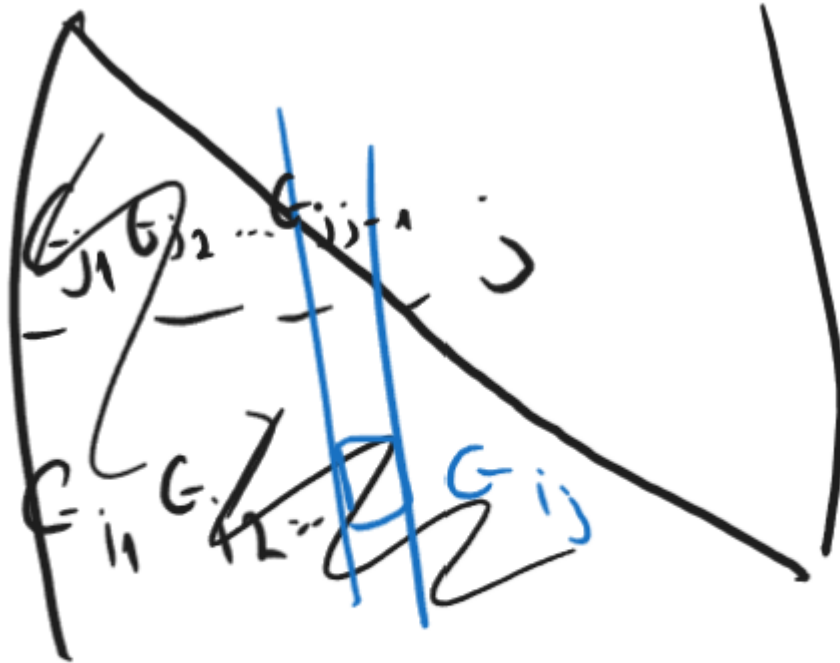
```

for j=1:N
    for i=j:N
        S_{ij} = A_{ij} - sum_{k=1}^{j-1} G_{ik} G_{kj}^T

        if i=j then
            S_{jj} = G_{jj} G_{jj}^T #разложение Холецкого (делаем), G_{jj} - находим
        else
            S_{ij} = G_{ij} G_{jj}^T
            #это решение СЛАУ вида XU=B (X - G_{ij}, U - G_{jj}^T, B - S_{ij})
        endif
    endfor
endfor

```

- 1) $G_{i1}G_{i2} \dots G_{ij-1}$
- 2) $G_{j1}G_{j2} \dots G_{jj-1}$



$$1 \leq i \leq N$$

Пункт IIa

$$\begin{pmatrix} A_{11} & [A_{21}]^T \\ A_{21} & A_{22} \end{pmatrix} = \\ = \begin{pmatrix} G_{11} & \text{нули} \\ G_{21} & I_{n-\alpha} \end{pmatrix} \times \begin{pmatrix} I_{\alpha} & \text{нули} \\ \text{нули} & A \end{pmatrix} \times \begin{pmatrix} [G_{11}]^T & [G_{21}]^T \\ \text{нули} & I_{n-1} \end{pmatrix} = \\ = \begin{pmatrix} G_{11}[G_{11}]^T & G_{11}[G_{21}]^T \\ G_{21}[G_{11}]^T & \hat{A} + G_{21}[G_{21}]^T \end{pmatrix}$$

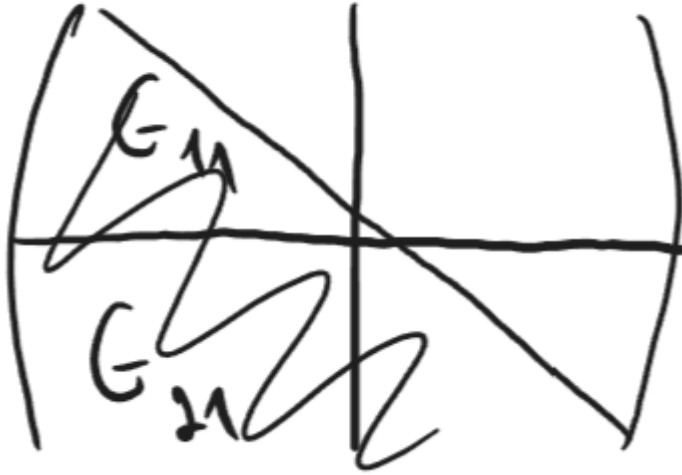
- 1) $A_{11} = G_{11}[G_{11}]^T$ - разложение Холецкого A_{11}
- 2) $A_{21} = G_{21}[G_{11}]^T$ - СЛАУ вида $XU = B$
- 3) $\hat{A} = A_{22} - G_{21}[G_{21}]^T - n = n - \alpha, A = \hat{A}$

Пункт IIб

- 1) α шагов Холецкого через гахру

находим G_{11} и G_{21}

- 2) $\hat{A} = A_{22} - G_{21}[G_{21}]^T - n = n - \alpha, A = \hat{A}$



Пункт II

- 1) α шагов Холецкого через модификацию матрицы внешним произведением - не трогая блок A_{22}
- 2) $\hat{A} = A_{22} - G_{21}[G_{21}]^T \cdot n = n - \alpha, A = \hat{A}$

Глава 4 Параллельные алгоритмы линейной алгебры

Лекция 11 (18.04.2026)

Параграф 4.1 Теоретическое и экспериментальное исследование параллельных алгоритмов

Пункт 4.1.1 Эффективность, ускорение, масштабируемость Ускорение (speedup): $S = \frac{T'}{T^p}$

T' - время работы последовательного

T^p - время работы параллельного, p - число задач

Смысл ускорения - во сколько раз вычисления по параллельному алгоритму производятся быстрее чем по последовательному

Эффективность (effecintly): $E = \frac{S}{p} = \frac{T'}{T^p p} * 100\%$

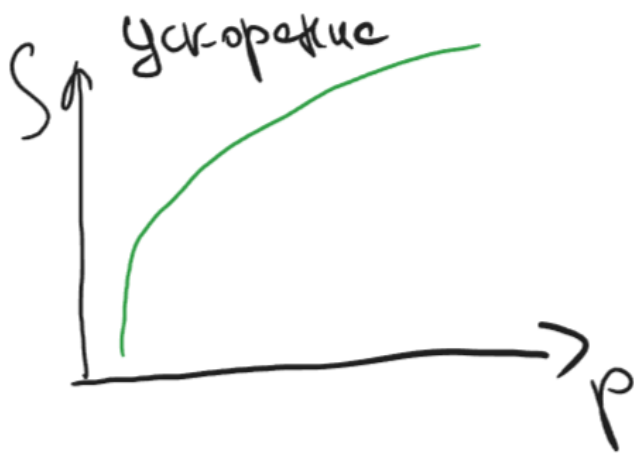
Смысл эффективности - указать какая доля исполнительных операций занимается полезными арифметическими операциями

Масштабируемость (scaleability): $M = \frac{V^p}{V'}$, при $T' = T^p$

V^p -объем памяти

V' - объем памяти при последовательном алгоритме

Смысл масштабируемости - это свойство алгоритма допускающее эффективное задействование большего количества исполнительных устройств с ростом объемов входных данных



Пункт 4.1.2

Пункт 4.1.2.1 Закон Амдала (1957) t' - время выполнения операций, которые не распараллелить

$\gamma = \frac{t'}{T'}$ - доля не распараллеленных операций в общем количестве

$$T^p = \gamma T' + \frac{1-\gamma}{p} T'$$

Тогда ускорение:

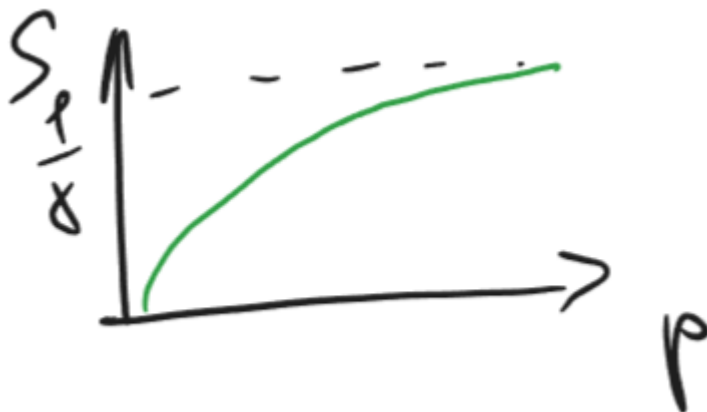
$$S = \frac{T'}{\gamma T' + \frac{1-\gamma}{p} T'} = \frac{1}{\gamma + \frac{1-\gamma}{p}}$$

Закон Амдала: $S \leq \frac{1}{\gamma + \frac{1-\gamma}{p}}$

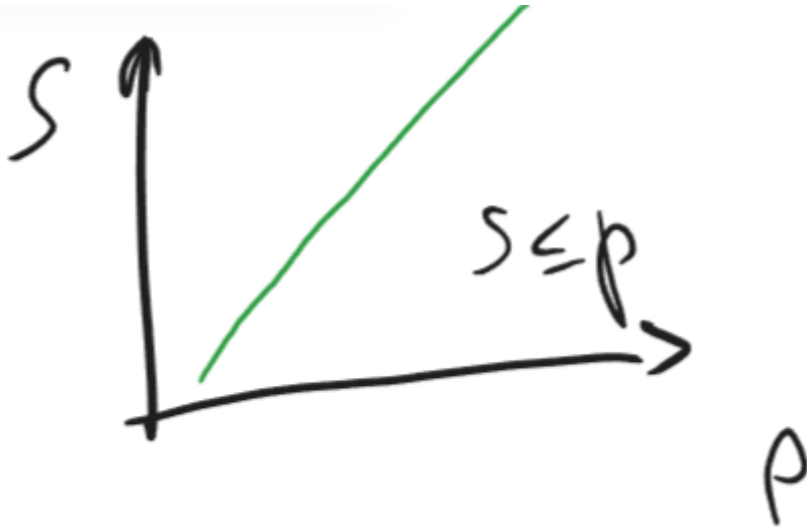
Рассмотрим крайности:

1. $p = \infty$

$$S \leq \frac{1}{\gamma}$$



2. $\gamma = 0$



Пункт 4.1.2.2 Учет коммуникаций Пусть изучаемый параллельный алгоритм сбалансирован, то есть совокупное время арифметических операций и коммуникаций для любой из его задач одинаково, начало и конец вычислений производится всеми задачами одновременно и для всех задач одинакова длительность арифметических операций и коммуникаций.

При этом в любой произвольный момент времени разные задачи могут делать разное.

Если мы накладываем и это требование (в любой момент времени все делают одно и то же, пусть и над разными данными), то алгоритм называют **систолическим**.

Любой **систолический** алгоритм всегда **сбалансирован**, обратное неверно.

Далее исключительно о сбалансированных алгоритмах.

t_a - общее время производства арифметических операций для любой задачи

t_k - общее время производства коммуникаций для любой задачи

Тогда:

$$S = \frac{t_a * p}{t_a + t_k} = \frac{p}{1 + \frac{t_k}{t_a}}$$

k - коэффициент сетевой деградаций

$$E = \frac{1}{1+k}$$

Распишем коэффициент сетевой деградации:

$$k = \frac{t_k}{t_a} = \frac{q \hat{t}_k}{Q \hat{t}_a}$$

q - количество коммуникаций

$\hat{t}_k$ - время одной

Q - количество арифметических операций

$\hat{t}_a$ - время одной

Разделим на две части коэффициент:

$\frac{q}{Q}$ - алгоритмическая часть коэффициента сетевой деградации

$\frac{\hat{t}_k}{\hat{t}_a}$ - аппаратная часть коэффициента сетевой деградации

Пункт 4.1.3 Экспериментальные исследования параллельных алгоритмов Если в ходе эксперимента производятся измерения физических величин, то он натурный;

Если расчет физических величин, то теоретический (вычислительный).

Ход исследования:

1. Поставить цель
2. Выбор параметров + обоснования
3. Выбор программной, аппаратной и системной базы
4. Предварительные теоретические предположения (либо вторым пунктом)
5. Постановка эксперимента (в итоге таблицы и графики) - **Необходим лабораторный журнал**
6. Описание результатов (словесное, а лучше письменное) - что происходит? как происходит?
7. Анализ (сравнение теории и эксперимента) - почему происходит?
 - 7а. Качественно и количественно соответствовать -> завершаем эксперимент
 - 7б. Качественно соответствовать, количественно не совпадать -> повод задуматься о выборе параметров, повторить или уточнить теорию
 - 7в. Никакого соответствия -> новая теория
8. Выводы (соответствие результата и цели)

Феноменологический подход

Лекция ?? (23.05.2026)

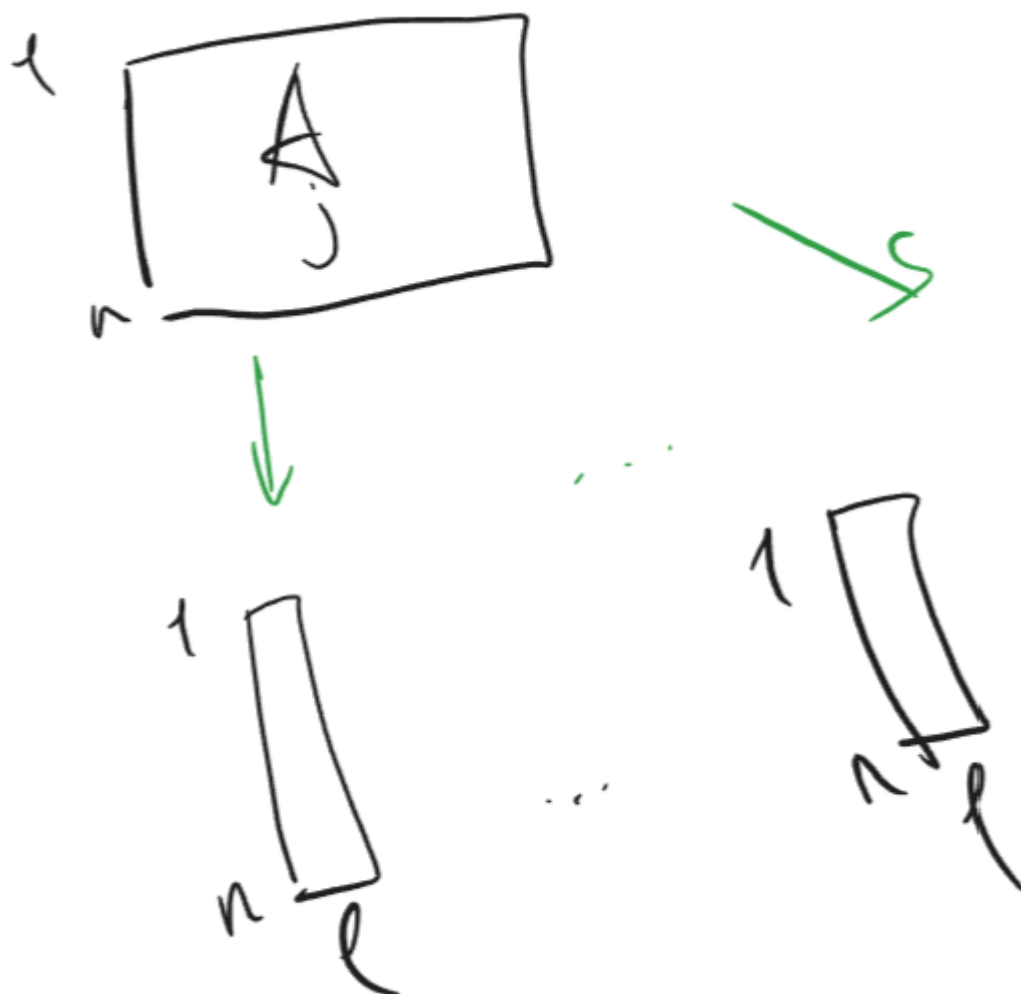
```
# инициализация
col = mu:p:n
L=length(col)
A_loc=A(:,1:L)
left, right
j = 1
l = 1
# локальный индекс столбца J подлежит вычислению в первую очередь любой задачей
# индекс столбца J только вычисляется или подлежит вычислению только
# в первую очередь той задачей в локальной памяти которой он хранится
# j у всех одинаковый, l может быть у всех различный
while l ≤ L then
    # произвольная задача не выйдет из цикла до тех пор
    # пока не вычислит все свои локальные столбцы,
    # в одной итерации цикла мы вычисляем j-ий столбец и используем его максимальным образом

    if j = col(l) then
        A_loc(j:n,l)=A_loc(j:n,l)*sqrt(A_lk(j,l))
        if j≠n then
            send(A_lk(j:n,l),right)
        endif

        for g=l+1:L
            r=col(q)
            A_loc(r:n,q)=A_loc(r:n,q)-A_loc(r:n,l)*A_loc(r,l)
        endfor
        j=j+1
        l=l+1
    else
        recv(g(j:n),left)
        if (right≠alpha) and (j<beta) then
            # alpha - номер задачи, породившая j-ий столбец
            # alpha можно не вычислять, а передавать по кольцу вместе с индексом j
            # beta - глобальный индекс последнего локального столбца правого соседа

            send(g(j:n),right)
        endif
        for q=l:L
            r=col(q)
            A_loc(r:n,q)=A_loc(r:n,q)-g(r:n)*g(r)
        endfor
        j=j+1
        l=l
    endif
endwhile
```

```
col_r = {M+1:p:n при mu<p; 1:p:n при mu>p}
l_r = length(col_r)
beta=col_r(L_r)
```



Алгоритм LU разложения кого то

```

# инициализация
col = mu:p:n
L=length(col)
A_loc=A(:,1:L)
left, right
k=1 #глобальный индекс
l=1
while l ≤ L then
    if k=col(l) then
        # 1. Расчет k-ого вектора множителей Гаусса
        # 2. Пересылка его правому соседу
        # 3. Производство модификации матрицы внешнего
        # произведения для локальных столбцов от l+1 до L
        k=k+1
        l=l+1
    else
        # 1. Принимаем k-ый вектор множителей Гаусса
        # 2. По возможности пересылаем правому соседу
        # 3. Производим модификацию матрицы внешним
        # произведением локальных столбцов от l до L
        k=k+1
    endif
endwhile

```